

SUBCHAT TREES: A SCALABLE HIERARCHICAL ARCHITECTURE FOR MULTI-THREADED DIALOGUE AND CONTEXT ISOLATION IN LLMS

MD MEHEDI HASAN MOON (SN. 202214048)
KABBO SARKER SURJO (SN. 202214030)
ARIFUL ISLAM KHAN (SN. 202214039)

A Thesis Submitted in Partial Fulfillment of the Requirements
for
the Degree of Bachelor of Science in Computer Science and
Engineering



DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING
MILITARY INSTITUTE OF SCIENCE AND TECHNOLOGY
DHAKA, BANGLADESH

JUNE, 2026

SUBCHAT TREES: A SCALABLE HIERARCHICAL
ARCHITECTURE FOR MULTI-THREADED DIALOGUE AND
CONTEXT ISOLATION IN LLMS

B.Sc. ENGINEERING THESIS

By

MD MEHEDI HASAN MOON (SN. 202214048)

KABBO SARKER SURJO (SN. 202214030)

ARIFUL ISLAM KHAN (SN. 202214039)

Approved as to style and content by the Examiners in June 2026:

Dr. Md. Aktaruzzaman

Associate Professor and Head

Department of RME, DIU

Supervisor

Brigadier General Mohammad Shahjahan Majib, ndu, psc

Department of Computer Science and Engineering (CSE)

MIST, Dhaka

Head of the
Department

SUBCHAT TREES: A SCALABLE HIERARCHICAL ARCHITECTURE FOR MULTI-THREADED DIALOGUE AND CONTEXT ISOLATION IN LLMS

DECLARATION

We hereby declare that the study reported in this thesis entitled as above is our own original work and has not been submitted before anywhere for any degree or other purposes. Further, we certify that the intellectual content of this thesis is the product of our own work and that all the assistance received in preparing this thesis and sources have been acknowledged and cited in the reference Section.

MD Mehedi Hasan Moon
Student No. 202214048

Kabbo Sarker Surjo
Student No. 202214030

Ariful Islam Khan
Student No. 202214039

ACKNOWLEDGEMENTS

The authors are thankful to Almighty Allah for His blessings during the completion of this thesis. The authors express their gratitude to their supervisor for guidance, feedback and encouragement throughout the research. The authors are also grateful to the Department of Computer Science and Engineering of Military Institute of Science and Technology for its support during the thesis work.

We thank the developers of ChromaDB, SentenceTransformers, and the Llama and Qwen model families for providing the foundational tools that enabled this research. We also acknowledge the LMSYS team for releasing the Chat-1M dataset (?), the Groq team for API access and the Kaggle platform for GPU compute resources.

ABSTRACT

SUBCHAT TREES: A SCALABLE HIERARCHICAL ARCHITECTURE FOR MULTI-THREADED DIALOGUE AND CONTEXT ISOLATION IN LLMS

Current LLM chat interfaces use linear transcripts to communicate with the user. In linear conversation every topic, sub-topic and even follow-up is appended to the same chronological history. This design is useful for short single-topic conversations but it becomes fragile in realistic long-term conversations. Where users switch between task/topic, return or mention earlier topic or ask nested follow-ups. Human conversations naturally follow a hierarchical structure but modern day AI interface lack conversational flow control in their interfaces. As a result, Unrelated context can leak across topics and older context can be evicted or buried. Current studies try to address this problem by increasing context length and models complexity. But Human conversation lacks its natural flow in chat interface. We see it as a design problem and propose Subchat Trees, a user-facing nested branching architecture for conversation management. Instead of treating the chat as one flat linear conversation, SubchatTree represents dialogue as a tree of parent and child subchats, where each node maintains its own local buffer, rolling summary and a shared vector-indexed long-term memory. A user can create a focused subchat from a selected parent excerpt, continue a side discussion in that branch and later return to parent without contaminating sibling or ancestor contexts. This external architecture is model-agnostic and can improve LLM performance without fine-tuning, as the model receives a cleaner branch-local context at inference time. The approach was evaluated on a controlled 353 turn Variable-Block Randomized Interleaving (VBRI) stress-test benchmark derived from five LMSYS-Chat-1M conversations, covering 59 topic labels and 43 topic-recall probes. Across Qwen2.5 7B, 14B and 32B models and buffer sizes $C \in \{5, 10, 20, 40\}$. SubchatTrees improves context-isolation F1 over the linear baseline in every model-buffer setting. The best performing model was Qwen32B at $C = 20$, with an F1 score of 85.2% and pollution rate was 16.2%. Recall probes also show stronger topic retention. Average ROUGE-1 improves over baseline in all tested settings, including 0.2476 to 0.4187 for 14B at $C = 20$ and 0.3338 to 0.4283 for 32B at $C = 20$. A reduction in input token per correct answer is also observed across all tested configurations. The results suggest that complex conversations need explicit external structure, not only longer context windows.

সারসংক্ষেপ

সাবচ্যাট ট্রিজ: এলএলএম-এ মাল্টি-থ্রেডেড সংলাপ ও প্রসঙ্গ বিচ্ছিন্নকরণের জন্য একটি স্কেলযোগ্য
স্তরবিন্যস্ত স্থাপত্য

বর্তমান বৃহৎ ভাষা মডেলভিত্তিক (LLM) চ্যাট ইন্টারফেসগুলো ব্যবহারকারীর সঙ্গে যোগাযোগের জন্য রৈখিক কথোপকথন-ইতিহাস ব্যবহার করে। রৈখিক কথোপকথনে প্রতিটি বিষয়, উপবিষয় এবং অনুসরণমূলক প্রশ্ন একই কালানুক্রমিক ইতিহাসে যুক্ত হয়। স্বল্পমেয়াদি ও একক-বিষয়ক কথোপকথনের ক্ষেত্রে এই নকশা কার্যকর হলেও বাস্তবসম্মত দীর্ঘমেয়াদি কথোপকথনে এটি দুর্বল হয়ে পড়ে, বিশেষত যখন ব্যবহারকারী কাজ বা বিষয় পরিবর্তন করেন, আগের কোনো বিষয়ে ফিরে যান বা স্তরবিন্যস্ত অনুসরণমূলক প্রশ্ন করেন। মানুষের কথোপকথন স্বাভাবিকভাবেই স্তরবিন্যস্ত; কিন্তু আধুনিক কৃত্রিম বুদ্ধিমত্তাভিত্তিক ইন্টারফেসে কথোপকথনের প্রবাহ নিয়ন্ত্রণের পর্যাপ্ত ব্যবস্থা নেই। ফলে একটি বিষয়ের সঙ্গে সম্পর্কহীন প্রসঙ্গ অন্য বিষয়ে প্রবেশ করতে পারে এবং পুরোনো প্রসঙ্গ সক্রিয় ইতিহাস থেকে বাদ পড়তে বা বিপুল তথ্যের মধ্যে চাপা পড়তে পারে। বর্তমান গবেষণাগুলো সাধারণত কনটেক্সটের দৈর্ঘ্য ও মডেলের জটিলতা বাড়িয়ে এই সমস্যা সমাধানের চেষ্টা করে। তবে শুধু এগুলো বাড়ালেই চ্যাট ইন্টারফেসে মানব কথোপকথনের স্বাভাবিক প্রবাহ বজায় থাকে না। আমরা সমস্যাটিকে একটি নকশাগত সীমাবদ্ধতা হিসেবে বিবেচনা করে কথোপকথন ব্যবস্থাপনার জন্য ব্যবহারকারীনির্ভর স্তরবিন্যস্ত শাখা-ভিত্তিক স্থাপত্য, Subchat Trees, প্রস্তাব করেছি। একটি সমতল রৈখিক কথোপকথন হিসেবে বিবেচনা না করে Subchat Trees সংলাপকে প্যারেন্ট ও চাইল্ড সাবচ্যাটের সমন্বয়ে গঠিত একটি ট্রি হিসেবে উপস্থাপন করে, যেখানে প্রতিটি নোড নিজস্ব স্থানীয় বাফার, ধারাবাহিক সারসংক্ষেপ এবং একটি অভিন্ন ভেক্টর-ইনডেক্সড দীর্ঘমেয়াদি মেমরি বজায় রাখে। ব্যবহারকারী প্যারেন্ট কথোপকথনের নির্বাচিত অংশ থেকে একটি নির্দিষ্ট বিষয়ে কেন্দ্রীভূত সাবচ্যাট তৈরি করতে পারেন, সেই শাখায় বিস্তারিত পার্শ্ব-আলোচনা চালিয়ে যেতে পারেন এবং পরে সমপর্যায়ের বা পূর্বসূরি প্রসঙ্গে অনাকাঙ্ক্ষিত তথ্য না মিশিয়ে মূল কথোপকথনে ফিরে আসতে পারেন। এই বাহ্যিক স্থাপত্যটি মডেল-নিরপেক্ষ এবং ফাইন-টিউনিং ছাড়াই LLM-এর কর্মদক্ষতা উন্নত করতে পারে, কারণ ইনফারেন্সের সময় মডেলটি সংশ্লিষ্ট শাখার তুলনামূলকভাবে পরিচ্ছন্ন ও প্রাসঙ্গিক কনটেক্সট পায়। প্রস্তাবিত পদ্ধতিটি পাঁচটি LMSYS-Chat-1M কথোপকথন থেকে তৈরি নিয়ন্ত্রিত 353টি টার্নের Variable-Block Randomized Interleaving (VBRI) স্ট্রেস-টেস্ট বৈধতার মূল্যায়ন করা হয়েছে, যেখানে 59টি বিষয়-লেবেল এবং 43টি বিষয়-স্মরণ যাচাইকারী প্রোব অন্তর্ভুক্ত রয়েছে। Qwen2.5 7B, 14B ও 32B মডেল এবং $C \in \{5, 10, 20, 40\}$ বাফার আকারের প্রতিটি পরীক্ষিত বিন্যাসে Subchat Trees রৈখিক বেসলাইনের তুলনায় কনটেক্সট-আইসোলেশন F1 উন্নত করেছে। সর্বোচ্চ কর্মদক্ষতা পাওয়া গেছে $C = 20$ -এ Qwen2.5 32B মডেলে, যেখানে F1 স্কোর ছিল 85.2% এবং কনটেক্সট দূষণের হার ছিল 16.2%। স্মরণ যাচাইকারী প্রোবগুলোও বিষয়ভিত্তিক তথ্য ধরে রাখার উন্নত সক্ষমতা দেখিয়েছে। সব পরীক্ষিত বিন্যাসে বেসলাইনের তুলনায় গড় ROUGE-1 বৃদ্ধি পেয়েছে; এর মধ্যে $C = 20$ -এ 14B মডেলের স্কোর 0.2476 থেকে 0.4187 এবং 32B মডেলের স্কোর 0.3338 থেকে 0.4283 হয়েছে। একই সঙ্গে, সব পরীক্ষিত বিন্যাসে প্রতিটি সঠিক উত্তরের জন্য প্রয়োজনীয় ইনপুট টোকেনের সংখ্যাও কমেছে। ফলাফলগুলো নির্দেশ করে যে জটিল কথোপকথন কার্যকরভাবে পরিচালনার জন্য শুধু দীর্ঘ কনটেক্সট

উইন্ডো যথেষ্ট নয়; এর পাশাপাশি সুস্পষ্ট বাহ্যিক কাঠামোও প্রয়োজন।

LIST OF MAIN NOTATION

LLM	Large Language Model
RAG	Retrieval-Augmented Generation
VBRI	Variable-Block Randomized Interleaving
F1	Harmonic mean of precision and recall
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
BLEU	Bilingual Evaluation Understudy
FIFO	First In, First Out
AWQ	Activation-Aware Weight Quantization
API	Application Programming Interface
SSE	Server-Sent Events
C	Local buffer capacity
B_i	Local buffer of conversation node i
T	Conversation tree
$\mathcal{V}$	Global vector memory

LIST OF TABLES

Table 3.1:	LLM temperature settings by task.	25
Table 4.1:	Conversation-level provenance for the final VBRI dataset. Turns denote replay turns contributed to <code>merged_realistic_interleaved.json</code> , excluding appended recall probes.	29
Table 4.2:	7B: merged context-isolation metrics across buffer sizes.	33
Table 4.3:	7B: aggregate recall-probe scores and probe cost. Topic-specific rows are kept in the appendix; this table reports the major aggregate recall results.	33
Table 4.4:	7B: merged system-performance metrics, including recall/summarization probe token accounting.	34
Table 4.5:	7B: merged recall/summarization-probe RAG decision breakdown.	34
Table 4.6:	14B: merged context-isolation metrics across buffer sizes.	35
Table 4.7:	14B: aggregate recall-probe scores and probe cost.	35
Table 4.8:	14B: merged system-performance metrics, including recall/summarization probe token accounting.	36
Table 4.9:	14B: merged recall/summarization-probe RAG decision breakdown.	36
Table 4.10:	32B: merged context-isolation metrics across buffer sizes.	37
Table 4.11:	32B: aggregate recall-probe scores and probe cost.	37
Table 4.12:	32B: merged system-performance metrics, including recall/summarization probe token accounting.	38
Table 4.13:	32B: merged recall/summarization-probe RAG decision breakdown.	38
Table 4.14:	Best Subchat Trees configuration for each model.	39
Table 5.1:	Complex Engineering Problem (CEP)	44

Table 5.2: Complex Engineering Activities (CEA)	44
Table A.1: Functional requirements of Subchat Trees.	48
Table A.2: Non-functional requirements and design responses.	48
Table A.3: Project timeline and work breakdown.	50
Table A.4: Resource and direct-cost budget for the research prototype.	50
Table A.5: Major project risks and mitigation measures.	50
Table A.6: Team roles and collaborative responsibilities.	51
Table A.7: Mapping of Knowledge and Attitude Profile attributes.	51
Table A.8: Mapping of Complex Engineering Problem attributes.	52
Table A.9: Mapping of Complex Engineering Activity attributes.	52
Table B.1: Mapping of attained CSE-400 course outcomes.	54

LIST OF FIGURES

- Figure 3.1: Overview of the SubchatTrees methodology showing four components: (a) hierarchical tree structure for conversation organization, (b) local buffer with fixed-size memory management, (c) global vector store for long-term archival and (d) multi-query RAG pipeline for context retrieval. 11
- Figure 3.2: Tree view of a Conversation Forest showing multiple conversation branches. Each node represents a subchat with isolated context. 13
- Figure 3.3: Three-tier memory design showing the relationship between the LocalBuffer (circular deque), rolling summarization, and vector store archival. 15
- Figure 3.4: End-to-end retrieval pipeline. **Phase 1:** LLM-based sub-query generation decomposes the user query into up to $n \leq 8$ sub-queries. **Phase 2:** each sub-query is embedded via `all-mpnet-base-v2` ($d=768$) and dispatched to ChromaDB for cosine search ($N=20$ candidates each), then deduplicated to $u=5$ unique results per sub-query. **Phase 3:** all anchor candidates are re-ranked by cosine similarity to the original query q_0 ; top- k are selected. **Phase 4:** each winning anchor expands $\pm\delta$ turns ($\delta=2$) into a coherent conversational block (Eq. 3.7). The resulting k blocks are injected into the LLM prompt. 19

- Figure 3.5: Context assembly pipeline. The user query is buffered (①–②) and passed through the RAG retrieval pipeline (③; detailed in Figure 3.4), which returns coherent context blocks R . At step ④ the pipeline branches: root-level nodes follow the *standard* path (solid arrows, Eq. 3.9), while follow-up subchat nodes additionally inject the selected-text prompt F and the inherited parent summary $S_{\text{sum}}^{\text{parent}}$ (dashed arrows, Eq. 3.10). Both paths converge at step ⑤ for LLM generation, after which the response is stored back in the buffer and vector store (⑥). 22
- Figure 3.6: User interface: (left) the conversation forest tree view with expandable nodes, and (right) the chat panel with text selection for one-click subchat creation. 25
- Figure 4.1: Context-isolation F1 across model sizes and buffer sizes. Subchat Trees improves F1 over the linear baseline in every configuration. 39
- Figure 4.2: Pollution rate across buffer sizes. Lower values are better. Subchat Trees consistently reduces cross-topic contamination by routing turns into branch-local buffers instead of keeping all topics in one linear transcript. 40
- Figure 4.3: Average ROUGE-1 F1 on 43 topic-recall probes. The tree system retains more semantically relevant topic content than the linear baseline across all tested model-buffer settings. 40
- Figure 4.4: RAG retrieval rate during the 43 recall probes. Probe retrieval is used as a memory-sufficiency signal because every probe asks the model to recover earlier topic content. 41
- Figure 4.5: Tokens per correct answer across model sizes and buffer sizes. The y-axis uses a log scale because the 7B baseline becomes extremely inefficient at larger buffers. 42

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	i
ABSTRACT	ii
সারসংক্ষেপ	iii
LIST OF NOTATIONS	v
LIST OF TABLES	vi
LIST OF FIGURES	viii
TABLE OF CONTENTS	x
1 INTRODUCTION	1
1.1 Research Background	1
1.2 Problem Statement	2
1.3 Overview of Subchat Trees	2
1.4 Thesis Objectives	3
1.5 Methodological Overview	3
1.6 Scope of The Thesis	3
1.7 Organization of The Thesis	4
2 THEORETICAL BACKGROUND AND LITERATURE REVIEW	5
2.1 Background: End-to-End Dialogue Modelling	5
2.2 Structured Reasoning and Branching Search	5
2.3 Context, Memory, and Retrieval	6
2.4 Conversation Structure and Disentanglement	7
2.5 Multi-Turn Interaction and Role-Based Structure	7
2.6 Research Gap	7
3 METHODOLOGY	9
3.1 System Architecture	9
3.1.1 Hierarchical Conversation Tree Structure	11

3.1.2	Memory Management Strategy	13
3.1.3	Context Inheritance	15
3.1.4	Follow-up Mechanism	16
3.2	Vector Memory and RAG Pipeline	16
3.2.1	Multi-Query Decomposition and Retrieval	16
3.2.1.1	Sub-Query Generation.	17
3.2.1.2	Embedding and Vector Search.	17
3.2.1.3	Deduplication.	17
3.2.2	Context Window Retrieval	19
3.2.3	Context Assembly	20
3.2.3.1	Standard Assembly (all nodes).	22
3.2.3.2	Follow-up Assembly (subchat nodes).	22
3.3	User Interface	23
3.3.1	Design Rationale.	23
3.3.2	The Problem with Linear Chat.	23
3.3.3	Subchat Trees as a Solution.	23
3.4	Baseline System	25
3.5	System Development	25
4	RESULT ANALYSIS	26
4.1	Dataset Design	26
4.2	Evaluation Matrices	28
4.3	Topic-Scoring Pipeline	31
4.4	How to Read the Result Tables	32
4.5	7B Model: Context Structure Helps, but Instruction Following Remains Fragile	33
4.6	14B Model: The Cleanest Practical Tradeoff	35
4.7	32B Model: Strongest Absolute Performance and Better Long-Context Tolerance	37
4.8	Cross-Model Visual Synthesis	39
4.9	Result Summary	41
5	ENGINEERING CONSIDERATIONS, CHALLENGES AND REMEDIES	43
5.1	Design Constraints	43
5.1.1	Context and Model Constraints	43

5.1.2	Memory and Retrieval Constraints	43
5.1.3	User-Controlled Branching Constraints	43
5.2	Ethical Considerations	43
5.3	Sustainability in Environmental and Societal Context	43
5.4	Complex Engineering Problem	44
5.5	Engineering Activities	44
5.6	Development Challenges and Remedies	44
5.7	Deployment Considerations	45
6	DISCUSSION AND CONCLUSION	46
6.1	Thesis Outcome	46
6.2	Thesis Implication and Contribution	46
6.3	What the Results Show	46
6.4	Why Larger Buffers Are Not Always Better	46
6.5	Token Efficiency Revisited	46
6.6	RAG as a Backstop Rather Than a Default	46
6.7	Limitations	47
6.8	Deployment Considerations	47
6.9	Conclusion	47
6.10	Future Work	47
6.11	Impact	47
A	ENGINEERING CONSIDERATIONS, CHALLENGES AND REMEDIES	48
A.1	System Requirements and Specifications	48
A.1.1	Relevant Specifications and Engineering Conventions	49
A.2	Design Constraints	49
A.2.1	Ethical Considerations	49
A.2.2	Environmental Considerations	49
A.2.3	Social Constraints	49
A.2.4	Sustainability in Environmental and Societal Context	49
A.3	Project Management	49
A.3.1	Timeline and Work Breakdown	49
A.3.2	Resource and Budget Management	50
A.3.3	Risk Management	50
A.3.4	Team Roles and Collaboration	51

A.4	Addressed Knowledge Profile Attributes	51
A.5	Addressed Complex Engineering Problems	51
A.6	Addressed Complex Engineering Activities	52
B	THESIS SELF-ASSESSMENT REPORT	53
B.1	Research Information	53
B.2	Synopsis of the Research	53
B.3	Mapping of Attained Course Outcomes	53
B.4	Assessment Questions and Answers	55
B.5	Signature Panel	56

CHAPTER 1

INTRODUCTION

1.1 Research Background

Large language models now serve as the backbone of chat systems used for programming assistance, education, planning, research and open-domain dialogue. Despite this advancement, failures remain common in long, multi-topic conversations. Earlier constraints are forgotten, separate task become mixed and outdated turns keep influencing later responses. This problem persists because most chat interface still organize every turn into a single chronological transcript, even when the underlying interaction contains multiple partially overlapping threads. As session grows, this chronological structure increase context pollution and makes it harder for the model to recover right information at the right time (?, ?, ?).

Recent studies suggest that explicit structure can improve language-model performance. Studies including Chain-of-Thought, Self-Consistency, Tree-of-Thought and Graph-of-Thought show that structured intermediate reasoning can improve LLM problem-solving ability on complex tasks. Conversation disentangle studies show that mixed message streams naturally contain distinct threads, but separating this thread after the fact is difficult and error-prone (?, ?, ?, ?). Other studies such as Hierarchical dialogue models, memory systems, retrieval augmentation and role-based interaction frameworks also introduce structure into how conversations are represented, stored, retrieved and managed (?, ?, ?, ?, ?, ?).

More recent work shows that multi-turn interaction is not simply repeated single term prompting. It is treated as a separate challenge that requires models to track users context, state and intent over time. This can be observed for multi-agents systems as well. Long-context studies show that relevant evidence becomes harder to use when it is buried in the middle of a input sequence (?, ?). Dialogue-focused studies also observed similar degradation pattern over long-running interaction such as, failures in factual consistency, event recall and context tracking (?, ?, ?, ?, ?). Context compression can reduce token cost and improve information density, but again this can compress multiple distinct threads into a single one (?, ?).

Consider a user who starts with Python debugging, shifts to python snakes for a biology question, opens a side discussion about Linux audio, and later returns to the original code. In a linear transcript, the latest query is interpreted against an interleaved history where keywords, pronouns, and instructions now refer to multiple competing contexts. The result can be semantic drift, role confusion, privacy leakage between topics and unnecessary token growth from carrying a broad history that no longer belongs to the current task.

The central issue, is not only how to store more context, but how to organize them so that unrelated conversational threads stop competing with one another. *SubchatTrees* address this problem at the interaction layer. Rather than asking the model to untangle a mixed transcript, the interface preserves topic structure explicitly through branch creation, local buffer, rolling summaries and branch-scoped retrieval. The goal is to keep active context focused, while still allowing long-term information to remain accessible.

1.2 Problem Statement

Most LLM chat interfaces store all messages in one chronological conversation. When users move between topics, return to previous tasks or create nested follow-ups, unrelated messages remain in the same context. The model must then decide which earlier messages are relevant while also following the current instruction. This creates context pollution, topic confusion and unnecessary processing of unrelated conversation history. This thesis addresses this interface and context-management problem through explicit hierarchical conversation branching.

These limitations become more visible as conversations grow in length and complexity. LLMs often struggle when reasoning and memory are not explicitly structured or managed. Long, unsegmented contexts are difficult for models to use reliably, even when the required information remains somewhere within the context window. Real-world conversations also contain multiple overlapping threads, and separating them after they have already been mixed is difficult and error-prone. Similar difficulties appear in multi-agent conversations, where the absence of clear roles and interaction protocols can lead to weak coordination and unclear conversational state. Together, these problems show the need for a conversation structure that can preserve topic boundaries, manage memory and support focused interaction throughout a long-running dialogue.

1.3 Overview of Subchat Trees

SubchatTrees organizes a session as a tree of related dialogue nodes rather than a single linear transcript. The design goal is to keep each active branch semantically focused while still allowing information to grow and persist through summarization and retrieval. The architecture centers on four design elements:

- Each conversation node maintains isolated context through local circular buffers (C messages) and vector-indexed long-term memory.
- Users create focused “subchats” by selecting specific text fragments from parent conversations.

- Follow-up prompts inject selected context as system messages, guiding the LLM’s attention.
- Parallel reasoning threads coexist without interference, and rolling summarization preserves semantic continuity beyond the buffer horizon.

1.4 Thesis Objectives

1. To minimize context pollution in long, multi-topic conversations by separating unrelated threads through chat interface.
2. To enable isolated and focused sub-conversations (subchats) where users can pursue side discussions without contaminating parent or sibling contexts.
3. To implement scalable memory management through a tree-structured conversation architecture that combines local buffers, rolling summaries and retrieval-based long-term memory.
4. To reduce latency and token cost while maintaining conversation quality.

1.5 Methodological Overview

The research was conducted by developing two conversation systems and evaluating them under the same conditions. The baseline stores every topic in one linear conversation node. Subchat Trees organizes the same conversation into parent and child nodes with isolated local buffers, rolling summaries and shared vector memory. A Variable-Block Randomized Interleaving benchmark was used to replay long multi-topic conversations across Qwen2.5 7B, 14B and 32B models with buffer sizes of 5, 10, 20 and 40. The systems were then compared using context-isolation, topic-recall, system-performance and recall-probe RAG metrics.

1.6 Scope of The Thesis

This thesis focuses on text-based long multi-topic conversation architectural and user interface design where users explicitly create and navigate subchats. It evaluates one model family across three model sizes and four fixed buffer sizes. The study focuses on context isolation, topic retention, retrieval behavior, token use and latency. Automatic branch creation, multimodal conversation and cross-model generalization are outside the current experimental scope.

1.7 Organization of The Thesis

The remainder of this thesis is organized as follows. Chapter 2 presents the theoretical background, related work and research gap. Chapter 3 describes the proposed architecture, memory design, retrieval pipeline, user interface and implementation. Chapter 4 presents the evaluation framework, metrics, experimental setup and results. Chapter 5 discusses engineering considerations, constraints, challenges and remedies. Finally, Chapter 6 presents the discussion, thesis outcomes, limitations, conclusion, future work and impact.

CHAPTER 2

THEORETICAL BACKGROUND AND LITERATURE REVIEW

Modern work on multi-turn LLM systems builds on a longer line of sequence modeling and neural dialogue research. Early encoder-decoder models showed that neural network could learn to map an input sequence to an output sequence (Vaswani et al., 2017). The transformer based attention mechanism reduced the bottleneck of fixed-length sequence representations (Vaswani et al., 2017), becoming the foundation of modern large-language models (Vaswani et al., 2017). Later, autoregressive models began to exhibit broad in-context task adaptation, like GPT-3’s few-shot results (OpenAI, 2020). Further, Instruction tuning and alignment work made this model practical task-following assistants by improving zero-shot generalization and their ability to follow user intent (Ouyang et al., 2022). Against that background, current studies are more focused on how context should be organized, retrieved and controlled during interaction.

2.1 Background: End-to-End Dialogue Modelling

Before the current wave of instruction-following LLM systems, dialogue modelling was often approached through end-to-end generative architectures trained on conversational corpora. Early studies like, Hierarchical recurrent encoder-decoder models (HRED) introduced a two-level structure in which token sequences formed utterances and utterances formed conversations (Sutskever et al., 2014). This work established an early argument that dialogues can be benefitted from explicit hierarchical organization rather than flat sequence processing alone.

Though the hierarchy remains internal to the model rather than exposed in the interaction design. But it proves that structure can improved model’s performance.

2.2 Structured Reasoning and Branching Search

Chain-of-Thought prompting shows that exposing intermediate reasoning steps can significantly improve models performance on complex reasoning tasks that require multi step inference (Wei et al., 2022). Self-Consistency extends this idea by sampling multiple reasoning paths and aggregating them rather than trusting a single chain (Wang et al., 2022). Tree-of-Thought and Graph-of-Thought push the same idea further by representing reasoning as a branching or graph-structured search process rather than a single linear derivation (Yao et al., 2023).

Taken together these studies suggest a broader conclusion. Structured representation and proper context engineering can improve models robustness, interpretability and search efficiency. Though their scope is limited to single tasks or single prompts, but subchat trees adopts the same high-level intuition about branching and decomposition. Though it is applied to persistent interaction structure rather than only to reasoning traces.

2.3 Context, Memory, and Retrieval

The limitations of long multi-turn conversational history are now well establish. Phenomenon such as “Lost in the middle” shows that language models exhibit strong position bias. Often struggle to utilize evidence placed in the middle of a conversation or prompt (?, ?). LiMuT extends this concern from static prompts to multi-turn dialogue. The study proves factual consistency degrades as conversations grow and the model must preserve earlier details across many exchanges just to be relevant (?, ?). These findings help explain why simply increasing context length does not reliably solve long-running dialogue problems.

Long-term dialogue benchmarks reach similar conclusions from the generation side. work such as “Beyond Goldfish Memory” studies multi-session open-domain conversation and finds that models trained on short exchanges struggle when dialogue depends on information learned across sessions (?, ?). “Long Time No See!” shows that long-term persona memory remains difficult to maintain without explicit memory management (?, ?). LoCoMo provides a large benchmark for very long-term conversational memory and reports that even long-context and RAG-enabled systems remain far from human performance on dialogue-grounded recall and event tracking (?, ?). Stateful memory-augmented transformers likewise target efficient retention of dialogue history without replaying the entire conversation at each turn (?, ?).

Memory and retrieval studies provide a relatable but different direction. Retrieval-Augmented Generation (RAG) improve language model performance by adding relevant external document to the prompt, instead of relying only on models internal parametric memory (?, ?). MemGPT treats long-context management as a hierarchical memory problem, where information can move between short term and long-term archival memory (?, ?). LongLLMLingua improves efficiency by compressing context while preserving relevent information (?, ?). Though these approaches can improve retrieval, memory management and prompt efficiency. However, the assumption of conversation organized as a single interaction threaded persist. SubchatTrees on the other hand, not only focuses on how conversation should be stored or retrieved, but also how conversation itself should be divided into seperate branches.

2.4 Conversation Structure and Disentanglement

Conversation disentanglement research studies how multiple threads become interleaved inside a single shared message stream. Elsner and Charniak introduced an early corpus and algorithm formulation for recovering detached conversations from mixed chat logs (Elsner & Charniak, 1999). Jiang et al. (Jiang et al., 2017) later showed that representation-learning approaches improve thread recovery by modeling message similarly within local context windows. Kummerfeld et al. (Kummerfeld et al., 2018) substantially expanded the scale and quality of annotated data, Establishing disentanglement as a robust research area.

These works are closely related to the problem addressed in this study, but they differ in where the intervention occurs. Conversation disentanglement is mainly reactive, the system tries to recover structure after the message from different thread have already been mixed. However, prior work also shows that recovering clean thread after Interleaving has occurred remains difficult and imperfect. This limitation suggest that conversation structure should be preserved before topic mixing occurs. Thus, SubchatTrees treats branching as an explicit user action during interaction, allowing the system to maintain separate thread from the beginning so that they don't mix up in the first place.

2.5 Multi-Turn Interaction and Role-Based Structure

Recent work increasingly treats multi-turn interaction as a problem in its own right rather than a side effect of stronger single-turn prompting. LiMuT, LoCoMo, Beyond Goldfish Memory, and Long Time No See! all point to the same pattern: coherence, recall, and state tracking degrade over extended dialogue unless the system actively manages long-term dependencies (LiMuT et al., 2020; LoCoMo et al., 2020; Beyond Goldfish Memory et al., 2020; Long Time No See! et al., 2020). The implication is that multi-turn reliability depends not only on raw model capability, but also on how conversational state is organized and revisited across turns.

CAMEL adds a relatable perspective from multi-agent systems (Schick et al., 2020). By assigning roles and communication protocols, it shows that explicit interaction structure can improve coordination and task performance. Subchat Trees applies a similar principle to a single-user setting: instead of multiple agents, separate branches provide explicit structure for topic-specific continuations, follow-ups and parallel exploration.

2.6 Research Gap

The literature consistently shows that Language Models benefits from structured in reasoning, memory and interaction. Long unsegmented context are difficult to use reliably and mixed conversational threads are hard to disentangle after con-

tamination has already occurred. Yet current systems are largely depended on a linear chronological transcript as the default interface. Also, There is no such work found proposing a user-facing hierarchical conversation architecture that allows conversations branching, context isolation and parallel exploration of topics over same context. To manage conversation flow more effectively and provide better context to the LLM, a user-centric interaction design is needed rather than relying only on larger context windows or post-hoc recovery. That missing interaction layer constitutes the core research gap addressed by this study.

CHAPTER 3

METHODOLOGY

3.1 System Architecture

Figure 3.1 provides an overview of the four main components of SubchatTrees: 1.tree-based conversation structure, 2.local buffer management, 3.global vector memory, and 4.the RAG retrieval pipeline.

a Interactive Branching Interface
User-driven branching from selected parent text

Selection-Based Branching

- User highlights a text span in the parent node
- A focused child subchat is created from that span
- Follow-up prompt F is attached to guide the child
- Parent context is inherited once at creation

Tree Navigation View

- Root node represents the main conversation
- Child nodes represent focused follow-up threads
- Depth encodes lineage $r \rightarrow \dots \rightarrow n$
- Sibling branches remain context-isolated

Messenger-Style Interaction — Familiar chat interface with in-place branching and focused follow-ups

Branching preserves the parent thread while opening a new child context.

(a) User interface with text-selection-driven subchat creation

b Hierarchical Conversation Forest
Forest = $\{T_1, T_2, \dots, T_p\}$ of rooted conversation trees

Tree T = (N, E, r, B, V)

Node Semantics

- Each node stores title, parent, children, and path
- Each node owns a local buffer B_n
- Follow-up nodes store selected-text context
- Child nodes copy parent state once at creation
- Subsequent turns remain local to that branch

Def. 1. Each node $n \in N$ has local state, parent linkage, and branch-specific memory. Siblings do not share context; inheritance occurs only at child creation.

(b) Hierarchical tree structure for conversation organization

c Three-Tier Node Memory
Per-node short-, medium-, and long-term context

Tier 1 **Short-Term: Circular Buffer B_n**
FIFO deque · C messages (default 15) · Per-node isolated

Tier 2 **Medium-Term: Rolling Summary S_n**
 $S_n = \text{LLM}(S_{[n-1]} \oplus M_{\text{buffer}})$ · Triggered at C, 2C, 3C, ...

Tier 3 **Long-Term: Vector Archive V**
Immediate vector indexing with node_id, role, timestamp, and turn number

Prompt assembly: local buffer and rolling summary are always available
Retrieved blocks R are appended only when retrieval is triggered for the current query.

Context Inheritance (Eq. 4):
At child creation: selected-text prompt + parent summary + parent buffer copy
One-time inheritance · No sibling sharing · No backward propagation to the parent

(c) Local buffer with fixed-size memory and rolling summarization

d On-Demand Retrieval Pipeline
Long-term retrieval from the vector archive when local context is insufficient

Retrieval trigger: The system decides whether archived context is needed for the current query
If not triggered, generation proceeds using node-local memory only.

Phase 1 Decompose
Generate q, plus up to $n \leq 8$ focused reformulations of the user query

Phase 2 Search
Embed each sub-query with all-mpnet-base-v2 and search ChromaDB (N = 20 candidates/query)

Phase 3 Re-rank
Deduplicate the candidate pool
Re-rank against the original query and keep top-k anchors

Phase 4 Expand
Expand each anchor into a turn-based window $\pm \delta$ ($\delta = 2$) to form coherent context blocks R

Embedding model: all-mpnet-base-v2, d = 768

Output: a small set of coherent retrieved blocks R for prompt assembly

(d) Multi-query retrieval pipeline for on-demand long-term memory access

(d) Multi-query retrieval pipeline for on-demand long-term memory access

Figure 3.1: Overview of the SubchatTrees methodology showing four components: (a) hierarchical tree structure for conversation organization, (b) local buffer with fixed-size memory management, (c) global vector store for long-term archival and (d) multi-query RAG pipeline for context retrieval.

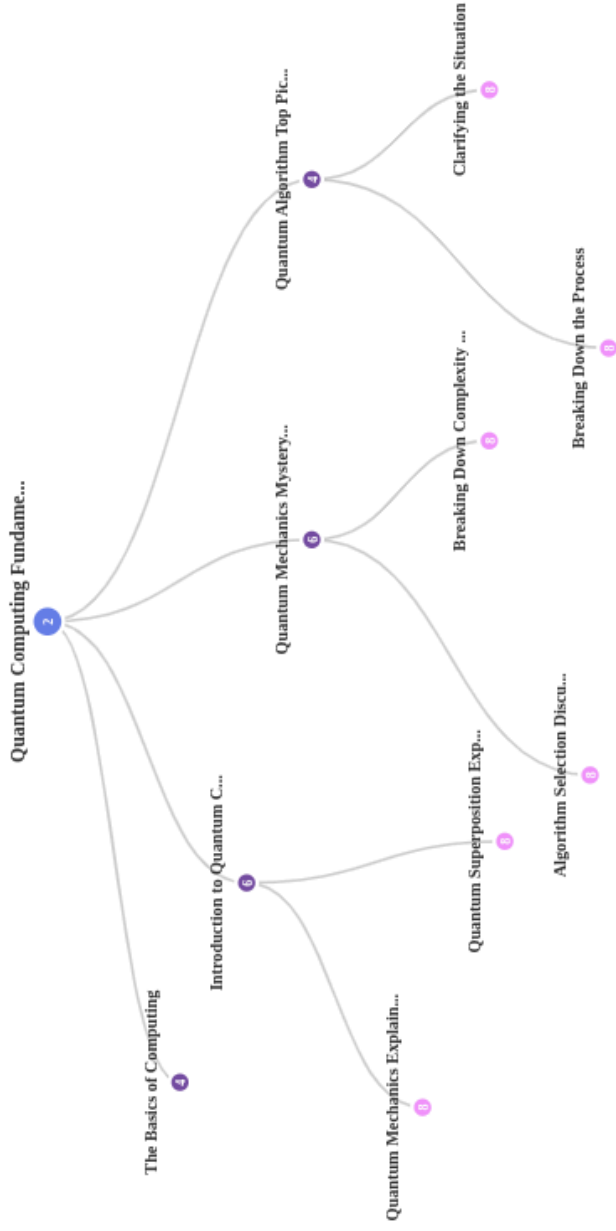
3.1.1 Hierarchical Conversation Tree Structure

The system organizes conversations as hierarchical tree structures, where each node represents a distinct conversation thread with isolated context.

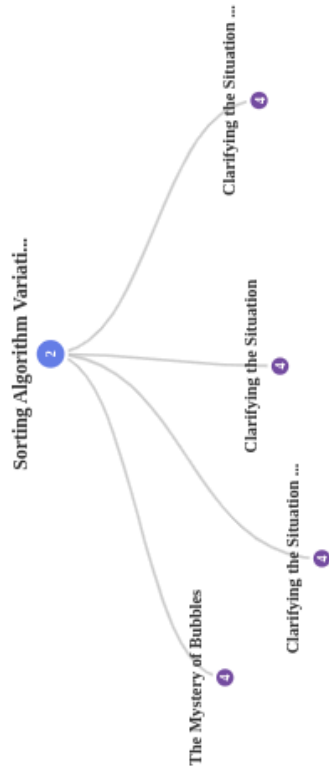
Definition 1 (Conversation Tree). *A conversation tree T is defined as a tuple of $(N, E, r, \mathcal{B}, \mathcal{V})$ here:*

- $N = \{n_1, n_2, \dots, n_k\}$ set of conversation nodes,
- $E \subseteq N \times N$ set of directed parent $\rightarrow$ child edges,
- $r \in N$ root node (main conversation),
- $\mathcal{B} : N \rightarrow B$ maps each node to its local buffer B_i ,
- $\mathcal{V} : N \rightarrow \mathbb{R}^d$ maps each node’s messages to d -dimensional embeddings space in a shared vector store.

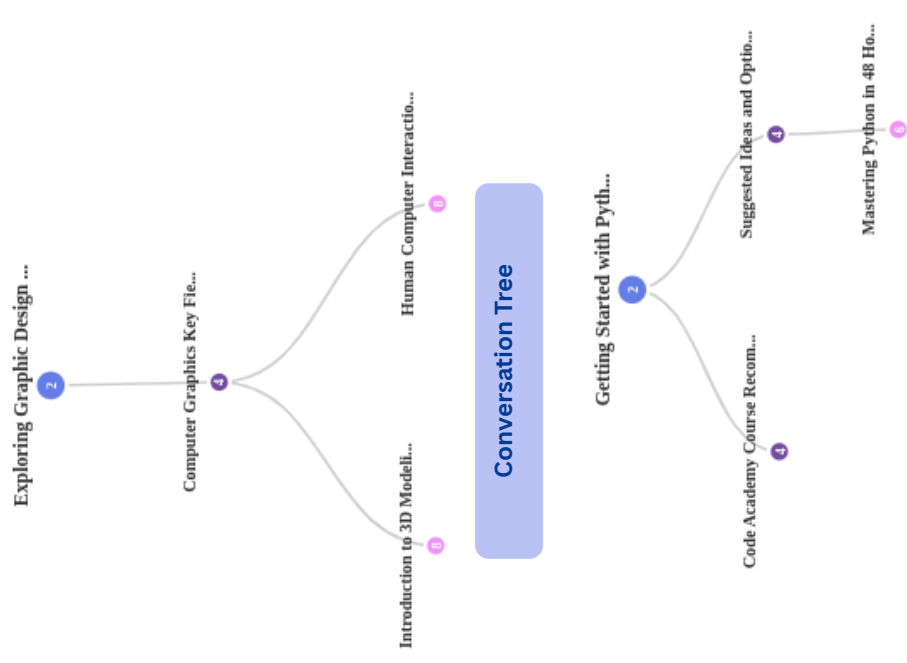
A **Forest** is a collection of independent trees $\mathcal{F} = \{T_1, T_2, \dots, T_p\}$, allowing users to maintain isolate conversation threads (e.g., work projects, personal queries, research topics) without any cross-contamination. Figure 3.2 illustrates Hierarchical organization of Conversational Trees, composing the **conversation Forest**.



Conversation Tree



Conversation Tree



Conversation Tree

Figure 3.2: Tree view of a Conversation Forest showing multiple conversation branches. Each node represents a subchat with isolated context. Each **Tree** node n_i maintains its own:

1. **Local Buffer** B_i : a circular deque with configurable capacity C (default $C=20$ messages).
2. **Node Metadata**: unique identifier (`node_id`), title, parent reference, children pointers.
3. **Follow-up Context**: user intent behind branch creation and context type (`follow_up`, `new_topic`, or `general`).

3.1.2 Memory Management Strategy

The memory architecture is divided into three complementary tiers, illustrated in Figure 3.3.

Hierarchical Memory Design & Context Assembly

Subchat Trees Architecture (Section 3.1 & 3.2)

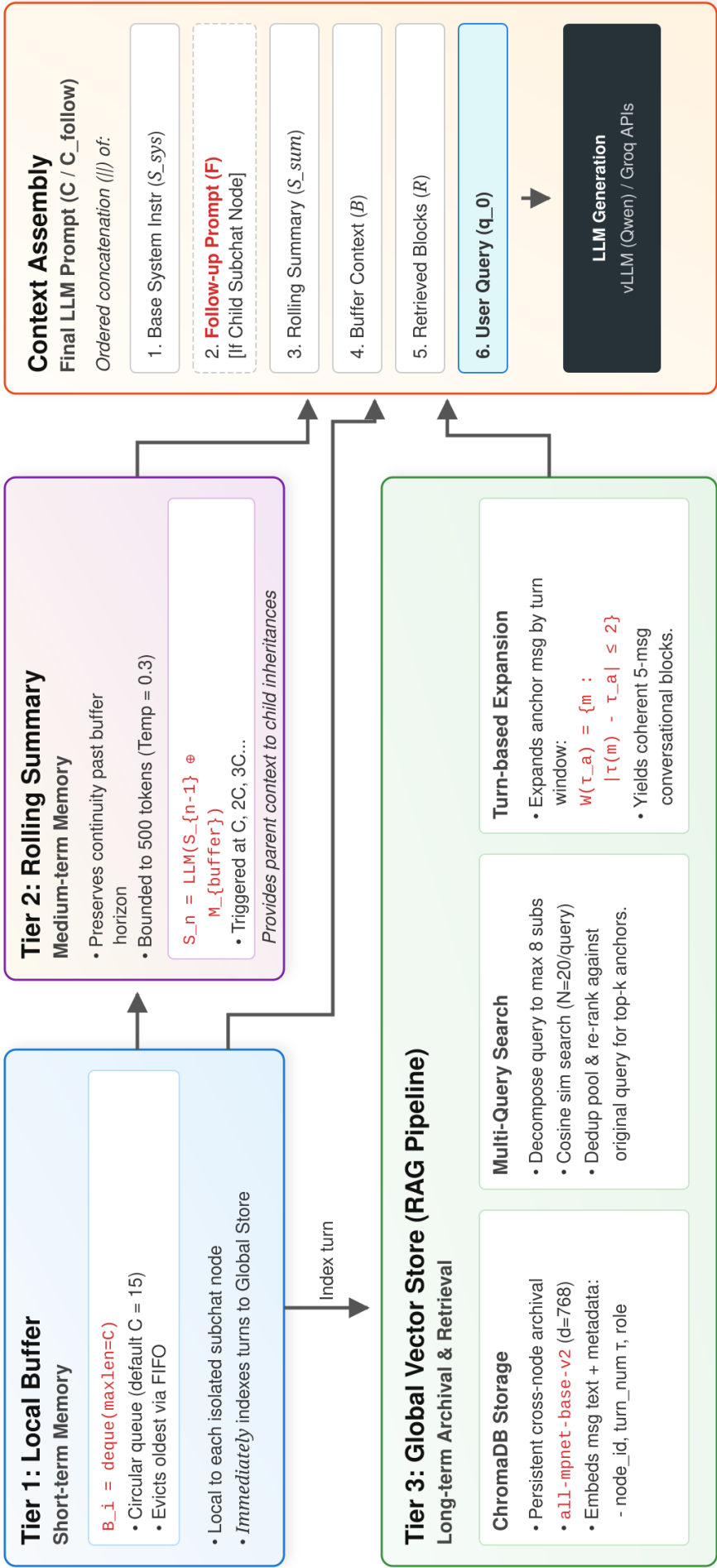


Figure 3.3: Three-tier memory design showing the relationship between the Local-Buffer (circular deque), rolling summarization, and vector store archival.

Tier 1: Local Buffer (Short-term Memory). Each node maintains a fixed-size circular buffer implemented as a `deque(maxlen=C)`:

$$B_i = \{(m_1, t_1), (m_2, t_2), \dots, (m_C, t_C)\} \quad (3.1)$$

where m_j is the j -th message, t_j its timestamp, and C the capacity. When $|B_i| > C$ or buffer limit reached, the oldest message is automatically evicted via FIFO policy. Crucially, messages are indexed to the global vector store *immediately* upon addition (not upon eviction), ensuring up to date information availability during buffer turnover or node switching.

Tier 2: Rolling Summarization (Medium-term Memory). To preserve semantic continuity beyond the buffer limit in long multi-turn conversation. A rolling summarization is triggered whenever the total message count reaches a multiple of the buffer size:

$$S_n = \text{LLM}_{\text{sum}}(S_{n-1} \oplus M_{\text{buffer}}), \quad \text{triggered at counts } C, 2C, 3C, \dots \quad (3.2)$$

where S_n represent current summary, S_{n-1} represent previous summary, M_{buffer} is the complete set of messages currently in the buffer and $\oplus$ denotes concatenation. This dynamic triggering adapts to any buffer size without hard-coded thresholds. The summary is bounded at 500 tokens and generated at temperature $T=0.3$.

Tier 3: Global Vector Store (Long-term Memory). All messages are embedded using `all-mpnet-base-v2` (SentenceTransformers, $d=768$) and stored in ChromaDB along with metadata:

$$\mathcal{V} = \{(\mathbf{e}(m), \text{meta}(m)) \mid m \in \text{all messages}\} \quad (3.3)$$

where $\mathbf{e}(m) \in \mathbb{R}^{768}$ is the embedding and $\text{meta}(m)$ includes `node_id`, `timestamp`, `role`, and `conversation_title`.

3.1.3 Context Inheritance

Creation of a child node n_c from parent n_p :

$$\text{Context}(n_c) = \text{FollowUpPrompt}(n_c) \cup \text{BufferCopy}(n_p) \cup \text{SummaryInherit}(n_p) \quad (3.4)$$

Specifically:

1. **Buffer Inheritance:** All messages from the parent’s buffer are copied into the child’s buffer. This provides immediate conversational context.
2. **Summary Inheritance:** The parent’s rolling summary is also inherited. Transferring long term context to child.

3. **Follow-up Prompt:** The selected text(parent’s fragment) and user’s intent (reason behind creating branch) are injected as a system message guiding the LLM’s attention.

The follow-up prompt is constructed as:

$$\text{FollowUpPrompt} = \text{``User selected: ''} \parallel s_{\text{sel}} \parallel \text{`` Focus on this.''} \quad (3.5)$$

Here, subsequent messages in the child subchat do *not* propagate back to the parent node, achieving bidirectional context isolation.

3.1.4 Follow-up Mechanism

Algorithm 1 formalizes the subchat creation process.

Algorithm 1 Follow-up Subchat Creation

Require: Parent node n_p , selected text s_{sel} , user query q , buffer size C

Ensure: Child node n_c with isolated context

```

1:  $n_c \leftarrow \text{NEWNODE}(\text{parent}=n_p, C)$ 
2:  $n_c.\text{context} \leftarrow (s_{\text{sel}}, \text{``follow\_up''})$ 
3:  $\text{sys\_msg} \leftarrow \text{BUILDFOLLOWUPPROMPT}(s_{\text{sel}})$ 
4: for each message  $m \in n_p.\text{buffer}$  do
5:    $n_c.\text{buffer.ADD}(m.\text{role}, m.\text{text})$  ▷ Inherit parent context
6: end for
7:  $n_c.\text{summary} \leftarrow n_p.\text{summary}$  ▷ Inherit rolling summary
8:  $r \leftarrow \text{LLM}(\text{sys\_msg} \parallel n_c.\text{buffer} \parallel q)$ 
9:  $n_c.\text{buffer.ADD}(\text{``user''}, q)$ 
10:  $n_c.\text{buffer.ADD}(\text{``assistant''}, r)$ 
11: return  $n_c$ 

```

3.2 Vector Memory and RAG Pipeline

The global vector store serves as long-term memory. It enables retrieval of relevant context from any point in any branch.

3.2.1 Multi-Query Decomposition and Retrieval

A single user query rarely captures all retrievable facets of a topic. Thus, the retrieval pipeline uses a four-phase process, 1. sub-query generation, 2. embedding-based vector search, 3. deduplication and 4. re-ranking. Figure 3.4 illustrates the full flow.

3.2.1.1 Sub-Query Generation.

We prompt the LLM (Groq API, temperature $T=0.3$) with intent-specific templates to decompose q into a set of sub-queries $Q = \{q_0, q_1, \dots, q_{n-1}\}$, where $q_0 = q$ is the original query and $q_1 \dots q_{n-1}$ are alternative query versions targeting related concepts, specific entities, or broader context ($n \leq 8$).

3.2.1.2 Embedding and Vector Search.

Each sub-query q_i is independently embedded using the `all-mpnet-base-v2` sentence-transformer model (,), producing a dense vector $e(q_i) \in \mathbb{R}^{768}$. Every indexed message in the system has been embedded with the same model and stored in a persistent ChromaDB (,) collection. Each stored message also includes metadata fields such as `node_id`, `role`, `timestamp`, `turn_number` τ , `title`, and an `archived` flag. A cosine-similarity search retrieves top $N=20$ candidate messages per sub-query from the collection.

3.2.1.3 Deduplication.

Multiple sub-queries can return the same message. Thus, the system deduplicate per sub-query (by content and message identifier), retaining at most $u = 5$ unique results each. A shared *seen* set further removes cross-query duplicates. The candidate pool is then:

$$\mathcal{P} = \bigcup_{i=0}^{n-1} R_i, \quad |R_i| \leq u, \quad |\mathcal{P}| \leq n \cdot u \quad (3.6)$$

where R_i denotes the unique results for sub-query q_i . A cosine-similarity re-ranking scores every candidate in $\mathcal{P}$ against the original query q_0 and returns the top- k anchor messages ($k \in [3, 10]$, default 5) for downstream turn-based expansion (Section 3.2.2).

Phase 1 — Multi-Query Decomposition

User Query q

"What languages have we discussed?"

LLM Sub-Query Generator

Groq API | Temperature = 0.3 | Intent-specific prompts

Output: $Q = \{ q_0 \text{ (original)}, q_1, q_2, \dots, q_{n-1} \} \ n \leq 8$

q_0 (original)

$N = 20$ results

q_1 (reformulation)

$N = 20$ results

q_2

$N = 20$ results

...

q_{n-1}

$N = 20$ results

Each sub-query dispatched in parallel to the vector store

Phase 2 — Embedding and Vector Search

Embedding Model: all-mpnet-base-v2

SentenceTransformers | 768 dimensions | Cosine similarity

$e(q_i) \in \mathbb{R}^{768}$

Each sub-query embedded independently

Index Path
add_message()

ChromaDB — Persistent Vector Store

Collection: "conversation_archive"

Metadata per message: node_id, role, timestamp, turn_number τ , title, archived

Deduplication

Per sub-query: keep $u = 5$ unique results (by text + message ID)

Cross-query: shared seen set filters global duplicates

Result: $\mathcal{P} = \bigcup R_i$ (up to $n \times u \approx 25$ unique anchor messages)

Phase 3 — Re-Rank Anchors

Cosine Re-Ranking Against Original Query q_0

Sort all ~ 25 anchors by $\text{cosine_sim}(e(\text{anchor}), e(q_0))$ descending

Tiebreaker: most recent timestamp first

Select Top-k Anchors

$k \in [3, 10]$, default $k = 5$

Phase 4 — Turn-Based Context Expansion

$\pm \delta$ Turn Window Expansion ($\delta = 2$)

For each anchor at turn τ_a : $W(\tau_a) = \{ m \in M_n : |\tau(m) - \tau_a| \leq 2 \} \cap \neg \text{Buffer}$

Neighbour messages scored at $0.8 \times$ anchor score | Dedup across overlapping windows

Output: Coherent Context Blocks

Block 1 (anchor at turn 15)

[turn 13] User: "What about R?"

[turn 14] Assistant: "R is popular in..."

[turn 15] User: "Python is great..." ← ANCHOR

[turn 16] Assistant: "Yes, Python leads..."

Block 2 (anchor at turn 42)

[turn 40] User: "How does Java..."

[turn 41] Assistant: "Java uses threads..."

[turn 42] User: "What about Go?" ← ANCHOR

[turn 43] Assistant: "Go uses goroutines..."

$R = k$ coherent blocks → injected into LLM prompt

Figure 3.4: End-to-end retrieval pipeline. **Phase 1:** LLM-based sub-query generation decomposes the user query into up to $n \leq 8$ sub-queries. **Phase 2:** each sub-query is embedded via `all-mpnet-base-v2` ($d=768$) and dispatched to ChromaDB for cosine search ($N=20$ candidates each), then deduplicated to $u=5$ unique results per sub-query. **Phase 3:** all anchor candidates are re-ranked by cosine similarity to the original query q_0 ; top- k are selected. **Phase 4:** each winning anchor expands $\pm\delta$ turns ($\delta=2$) into a coherent conversational block (Eq. 3.7). The resulting k blocks are injected into the LLM prompt.

3.2.2 Context Window Retrieval

A retrieved message in isolation often lacks the conversational context necessary for coherent understanding. A naïve approach would define a temporal window (e.g., ± 60 s around the anchor), but user response latencies are inconsistent and unpredictable—a reader may pause for seconds or minutes before replying—making time-based windows unreliable. We Therefore define a *turn-based* context window anchored on conversational sequence.

Each message m is indexed with a monotonically increasing turn number $\tau(m)$ within its node. For an anchor message with turn number τ_a , the context window is:

$$W(\tau_a) = \{m \in M_n : |\tau(m) - \tau_a| \leq \delta\} \quad (3.7)$$

where M_n is the set of all indexed messages in node n and $\delta = 2$ (i.e., two turns before and two turns after the anchor). This yields at most $2\delta + 1 = 5$ messages per anchor regardless of the time elapsed between turns. As with temporal windows, we exclude any message still held in the active buffer:

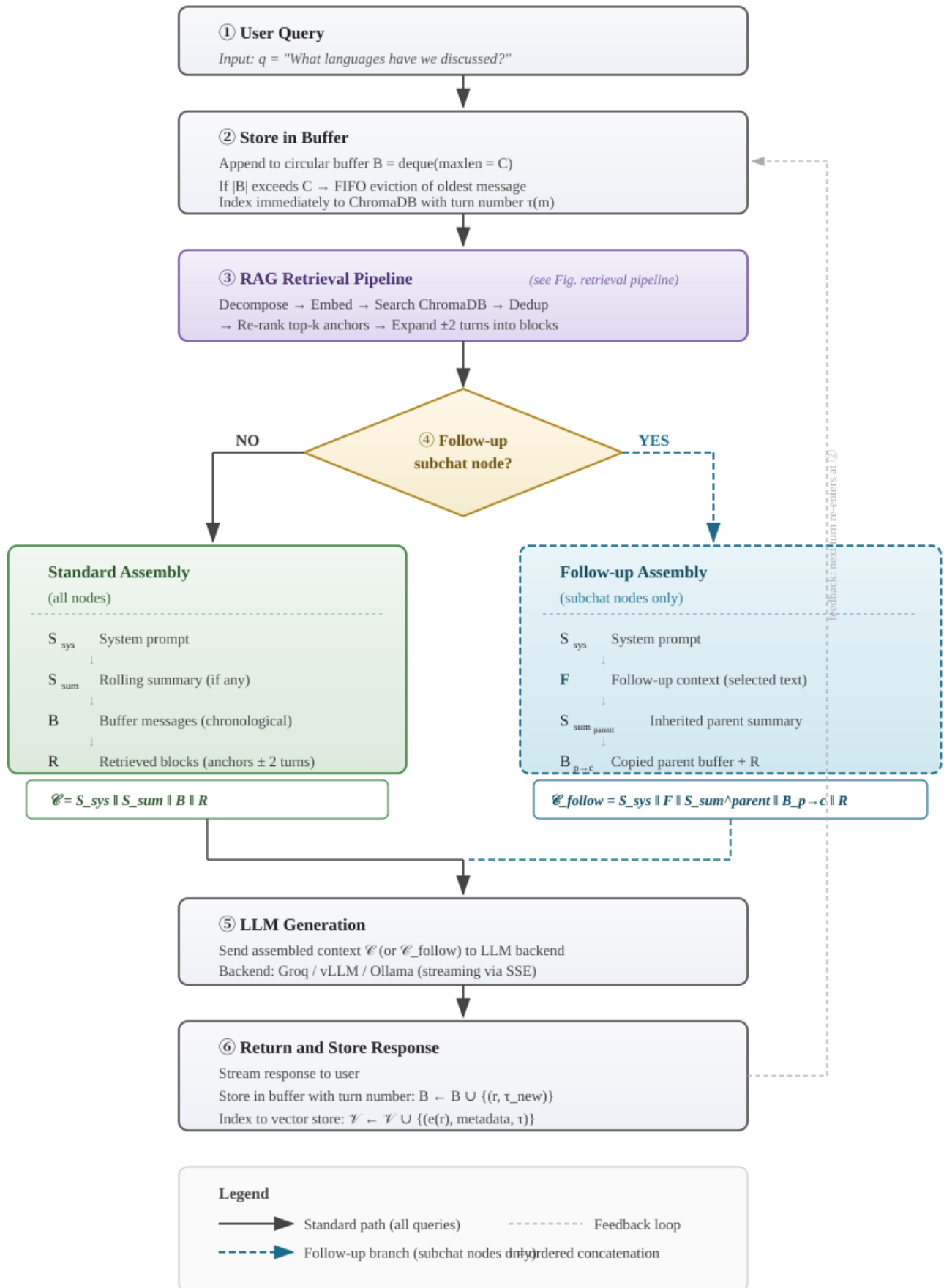
$$W_{\text{filtered}}(\tau_a) = W(\tau_a) \setminus B \quad (3.8)$$

here, B denotes the current buffer contents.

Context expansion is applied *after* re-ranking rather than before. The retrieval pipeline first re-ranks all deduplicated anchor candidates by cosine similarity to the original query q_0 and selects the top- k anchors ($k \in [3, 10]$, default 5). Only then does each winning anchor expand into its $\pm\delta$ turn window. This “*rank-then-expand*” ordering ensures that neighbouring messages never compete with anchors during ranking. Instead of that, each anchor is delivered to the LLM as a coherent conversational block rather than as scattered fragments interleaved with unrelated messages. Neighbour messages receive a discounted relevance score of $0.8 \times$ the anchor score. This helps preserving the distinction between directly matched content and its surrounding narrative.

3.2.3 Context Assembly

The final prompt sent to the LLM is an ordered sequence of messages. Its composition differs depending on whether the active node is a *root-level* conversation or a *follow-up subchat*. Figure 3.5 illustrates both paths.



Context assembly pipeline with follow-up branching. Retrieval details in Fig. retrieval pipeline.

Figure 3.5: Context assembly pipeline. The user query is buffered (①–②) and passed through the RAG retrieval pipeline (③; detailed in Figure 3.4), which returns coherent context blocks R . At step ④ the pipeline branches: root-level nodes follow the *standard* path (solid arrows, Eq. 3.9), while follow-up subchat nodes additionally inject the selected-text prompt F and the inherited parent summary $S_{\text{sum}}^{\text{parent}}$ (dashed arrows, Eq. 3.10). Both paths converge at step ⑤ for LLM generation, after which the response is stored back in the buffer and vector store (⑥).

3.2.3.1 Standard Assembly (all nodes).

Every query passes through the seven-step RAG pipeline described above. The assembled prompt takes the form:

$$\mathcal{C} = S_{\text{sys}} \parallel S_{\text{sum}} \parallel B \parallel R \quad (3.9)$$

where S_{sys} is the base system instruction, S_{sum} is the node’s rolling summary (omitted if none exists), B is the circular buffer (recent turns in chronological order), R is the set of retrieved messages returned by the RAG pipeline (empty when retrieval is not triggered), and $\parallel$ denotes ordered concatenation.

3.2.3.2 Follow-up Assembly (subchat nodes).

When a user selects text in a parent conversation and creates a subchat. The child node inherits three artifacts from its parent at creation time:

1. **User intent prompt.** The text the user highlighted, along with users intent, is wrapped in a system message directing the model to focus on that specific topic.
2. **Inherited summary.** The parent’s rolling summary is copied to the child, providing long term context.
3. **Parent buffer messages.** All messages currently in the parent’s circular buffer are copied into the child’s buffer. This ensures the child begins with the full recent context of the parent conversation.

The resulting prompt becomes:

$$\mathcal{C}_{\text{follow}} = S_{\text{sys}} \parallel F \parallel S_{\text{sum}}^{\text{parent}} \parallel B_{\text{parent} \rightarrow \text{child}} \parallel R \quad (3.10)$$

where F is the follow-up context, $S_{\text{sum}}^{\text{parent}}$ is the inherited parent summary, and $B_{\text{parent} \rightarrow \text{child}}$ denotes the child’s buffer initialized with the parent’s messages. As the child conversation progresses, new turns are appended to this buffer (evicting older ones via FIFO), and its own rolling summary gradually replaces the inherited one, ensuring continuity while allowing topic divergence. But parent’s state remain unchanged.

3.3 User Interface

3.3.1 Design Rationale.

The interface deliberately adopts a messenger-style layout—a familiar paradigm shared by WhatsApp, iMessage, and Slack—so that users can transfer existing mental models to the new system without additional learning overhead (?, ?). By anchoring interaction in a well-understood pattern (send a message, receive a reply), the cognitive cost of the novel *subchat* mechanism is kept low.

3.3.2 The Problem with Linear Chat.

Conventional LLM chat interfaces present all exchanges in a single, chronologically ordered thread. As conversations grow in length or branch across topics, this layout introduces several usability failures: (i) users must scroll extensively to relocate earlier context, (ii) there is no affordance for branching—returning to a prior topic forces the user to re-state context or risk contaminating the current thread, and (iii) the model’s fixed context window silently evicts earlier turns, creating an invisible boundary between what the user remembers and what the model can still “see.” These issues compound in exploratory or multi-topic sessions.

3.3.3 Subchat Trees as a Solution.

Our interface exposes the underlying tree structure through two complementary views (Figure 3.6): a collapsible *tree navigator* on the left and a focused *chat panel* on the right. Users create a subchat by selecting any text in the chat panel and clicking a single button; the new subchat inherits relevant context (Section 3.2.3) without disrupting the parent conversation. This design follows the principle of *progressive disclosure* (?, ?): the default view is a simple chat, but richer structure (branching, parallel follow-ups, hierarchical navigation) is available on demand.

oC You

what is python ?

InsightFlow

 Ask InsightFlow

Python is a high-level, interpreted programming language that is widely used for various purposes such as web development, scientific computing, data analysis, artificial intelligence, and more. It was created in the late 1980s by Guido van Rossum and was first released in 1991.

Key Features of Python:

1. **Easy to Learn:** Python has a simple syntax and is relatively easy to learn, making it a great language for beginners.
2. **High-Level Language:** Python is a high-level language, which means it abstracts away many low-level details, allowing developers to focus on the logic of the program without worrying about the underlying mechanics.
3. **Interpreted Language:** Python code is interpreted at runtime, which means that the code is executed line by line, rather than being compiled into machine code beforehand.
4. **Object-Oriented:** Python supports object-oriented programming (OOP) concepts, such as classes, objects, inheritance, and polymorphism.
5. **Dynamic Typing:** Python is dynamically typed, which means that the data type of a variable is determined at runtime, rather than being specified at compile time.
6. **Large Standard Library:** Python has a vast standard library that includes modules for various tasks, such as file I/O, networking, and data structures.
7. **Cross-Platform:** Python can run on multiple platforms, including Windows, macOS,

Ask anything...

InsightFlow can make mistakes. Consider checking important information.



Welcome to InsightFlow

Your intelligent partner in discovery. How can I assist you today?



Brainstorm ideas

for a new marketing campaign



Summarize a topic

on quantum computing



Compare and contrast

Next.js vs. React



Design a plan

for a trip to Kyoto

are python poisonous

InsightFlow can make mistakes. Consider checking important information.

Figure 3.6: User interface: (left) the conversation forest tree view with expandable nodes, and (right) the chat panel with text selection for one-click subchat creation. Key affordances include: (i) breadcrumb navigation showing the path from root to the active subchat, (ii) visual indicators for inherited context and active retrieval status, (iii) parallel follow-ups that let users explore multiple branches of a topic simultaneously, and (iv) real-time token streaming via Server-Sent Events across multiple LLM backends (Groq, vLLM, Ollama).

3.4 Baseline System

For comparison, we implement a linear chat baseline:

- Single conversation thread with a shared circular buffer (C messages, FIFO eviction).
- All topics share the same context window—no isolation mechanism.
- Identical LLM backend, embedding model, and temperature settings.

Both systems use the same buffer capacity C in each experiment, isolating the effect of conversation structure.

3.5 System Development

Table 3.1: LLM temperature settings by task.

Task	Temperature	Rationale
Main conversation	0.3	Deterministic replay
Title generation	0.3	Consistent naming
Summarization	0.3	Structured output
Topic-prefix scoring	—	Deterministic regex evaluation
Query decomposition	0.3	Structured retrieval

CHAPTER 4

RESULT ANALYSIS

4.1 Dataset Design

The final evaluation dataset is `VBRI_merged.json`, a 352-turn stress-test conversation derived from five real LMSYS-Chat-1M conversations (?, ?) produced by vicuna-13b, alpaca-13b, and koala-13b. Each source conversation was first transformed into a replayable hierarchical script with explicit topic labels, subchat actions, selected-text spans, and required response-prefix labels. The final benchmark then merges these five scripts into a single interleaved conversation.

Variable-Block Randomized Interleaving (VBRI). To stress-test context isolation and recall under realistic multi-topic switching patterns, we developed a merged evaluation scenario using Variable-Block Randomized Interleaving (VBRI). VBRI simulates sessions where users return to earlier topics after short bursts of unrelated discussion while preserving the temporal logic of each original conversation.

The VBRI algorithm operates as follows:

1. **Source-Level FIFO:** Each of the five source scenarios maintains a local queue of ordered blocks. On each iteration, the algorithm selects one source scenario and removes the next block from that source’s queue. This guarantees that the original temporal order within every source conversation is preserved.
2. **Variable Blocks:** Consecutive same-topic turns are grouped into blocks. Blocks longer than $b_{\max}=4$ are split into smaller ordered sub-blocks, preserving short local coherence while preventing one topic from dominating the merged stream.
3. **Temporal Preservation:** Within each source scenario, turns maintain their original sequential order. This preserves logical dependencies (e.g., follow-up questions must appear after their referents) while allowing cross-scenario interleaving.
4. **Topic Return Dynamics:** With probability $p_{\text{ret}}=0.3$, the next block is drawn from the same source as the previous block; otherwise, another active source is selected uniformly. This creates bursty topic revisitation rather than complete random shuffling.
5. **Deterministic Reproducibility:** A fixed random seed (42) ensures identical interleaving across experimental runs.

The resulting merged dataset contains 309 original conversation turns spanning 59 distinct LMSYS-derived topics. It contains 91 topic switches, 64 source-conversation

switches, 25 topic returns, and no temporal-order violations within any source conversation.

Summarization Probes. After the 309 ordinary turns, the dataset appends 43 summarization-probe steps, producing 352 total steps. These probes are added only for topics with at least three user turns. Each probe asks the model to summarize what was discussed about one topic, for example: `Summarize everything we have discussed about ai chat tools. Include all key points, details, and questions that were raised.` The purpose is to test topic-level context retention after a long interleaved conversation, not only immediate topic-prefix selection. In the linear baseline, every probe is sent to the same main conversation node, where all topics have been mixed into one shared history. In Subchat Trees, probes are routed to the relevant topic subchat branch, so the model retrieves from a branch whose local buffer, summary, and vector memory are focused on one topic. This design directly tests whether dividing roles across topic-specific nodes improves the model’s ability to recall earlier information.

Dataset Structure. The dataset is encoded as a JSON file with the following schema:

- `scenario_name`: Unique identifier for the scenario.
- `conversations`: Ordered list of conversation steps.
- Each step contains:
 - `step`: Integer turn number (1-indexed).
 - `context`: Ground-truth topic label (e.g., ```indian_history```, ```astrology```).
 - `message`: Raw user query.
 - `node_type`: Target subchat identifier (e.g., ```main```, ```subchat_indian_history```).
 - `action`: Operation type (`create_subchat`, `switch_node`, or implicit continuation).
 - `selected_text`: (Optional) Text fragment selected for follow-up subchat creation.
 - `subchat_title`: (Optional) Human-readable title for new subchats.
 - `parent_node_type`: (Optional) Parent node for hierarchical subchat creation.
 - `is_recall_probe`: (Optional) Boolean marker for appended summarization-probe steps.
 - `probe_topic`: (Optional) Topic being tested by a summarization probe.
 - `probe_target_node`: (Optional) Target node for a summarization probe.

For example, a step from the merged interleaved dataset:

```
{
  "step": 45,
  "context": "indian_legal_family",
  "message": "Can you explain the Hindu Succession Act?",
  "node_type": "subchat_indian_legal_family",
  "action": "create_subchat",
  "selected_text": "Indian legal framework",
  "subchat_title": "Hindu Succession Act Discussion",
  "parent_node_type": "subchat_indian_legal"
}
```

This structure enables deterministic replay: the test harness iterates through steps sequentially, performs the specified action (create subchat, switch node), sends the user message to the target node, and scores the response against the ground-truth context label.

Summarization probes use the same replay mechanism. They are encoded as `switch_node` steps so the baseline can ask the probe in its single main node, while Subchat Trees can ask the same probe in the topic-specific node indicated by `probe_target_node`. Probe responses are excluded from topic-prefix isolation counts and are used only for recall scoring.

In the baseline condition, all steps are replayed into one linear conversation node (MAIN) regardless of their annotated `node_type`. In the Subchat Trees condition, `create_subchat` steps instantiate child nodes under the annotated parent, `switch_node` steps route the next message to an existing node, and ordinary continuation steps remain in the current annotated node. This means both conditions receive the same user messages in the same order; only the context organization differs.

Table 4.1 summarizes the dataset provenance using the original LMSYS conversation identifiers. The table reports the five source conversations that form `merged_vbri_temporal`, the final dataset used in the Kaggle run.

The final replay contains 309 ordinary conversation turns and 59 normalized topic labels. Its replay annotations create 51 subchats. The probe-injected version appends 43 recall probes, producing 352 total steps; those probes are not counted as ordinary conversation turns in Table 4.1.

4.2 Evaluation Matrices

The evaluation reports four complementary matrices. Each matrix measures a different part of the study: whether a conversation system can keep the right topic active, retain the content of that topic, how efficiently the system is and can we reduce retrieval rate.

Context Isolation Matrix. The context isolation matrix measures turn-level topic control. For each ordinary replay turn t , the dataset provides a ground-truth topic

Table 4.1: Conversation-level provenance for the final VBRI dataset. Turns denote replay turns contributed to `merged_realistic_interleaved.json`, excluding appended recall probes.

Source ID	Conversation	Turns	Topics	Source Stress Pattern
22807e655dd04		52	4	Persona roleplay, role reversal, LLM-knowledge questions, and Twenty Questions
2348cb0ee402				game-state confusion
3672e70		62	11	Medical discussion, AI consciousness, repetitive-loop failure, pandemic safety, and literature analysis
6c4992f0aed04				DSP, C programming, medical genetics, Linux audio, physics, science fiction, geography, and games
dd3bf9a4bc225		77	17	
bb4fb0				
8d10c143f8fc4		53	8	Cookie recipes, recipe-halving math errors, argumentative correction, jokes, and emoji game turns
a7599a5a1877				
8fec112		64	19	Explicit context resets across Indian history, LSAT reasoning, law, nutrition, astrology, travel, therapy, and chess
eaf06f12a1d74				
e5ca30a7ca94				
a7c4128				
ec366fd3e4b54				
82e8acac750f				
9b3b55b				
merged_vbri_temporal		309	59	Final interleaved benchmark used for context-isolation scoring and recall probes

label τ_t . The model response is scored by extracting the leading topic prefix, denoted $\hat{\tau}_t$. A turn is correct when $\hat{\tau}_t = \tau_t$; otherwise it is counted as context pollution because the response has been assigned to the wrong conversational context. This metric therefore evaluates both memory of the active topic and the ability to match the user query to the correct topic state.

For each topic τ_i , we build a one-vs-rest confusion matrix:

- TP_{*i*}: τ_i was expected and the response was labeled as τ_i .
- FP_{*i*}: another topic was expected, but the response was labeled as τ_i .
- FN_{*i*}: τ_i was expected, but the response was labeled as another topic or as unknown.

The support n_i is the number of turns whose ground-truth topic is τ_i . Per-topic precision, recall, and F1 are:

$$P_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i}, \quad R_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FN}_i}, \quad F1_i = \frac{2P_iR_i}{P_i + R_i}. \quad (4.1)$$

The main table reports support-weighted averages:

$$\text{Precision}_w = \sum_i \frac{n_i}{\sum_j n_j} P_i, \quad \text{Recall}_w = \sum_i \frac{n_i}{\sum_j n_j} R_i, \quad F1_w = \sum_i \frac{n_i}{\sum_j n_j} F1_i. \quad (4.2)$$

Macro averages are also computed by giving every topic equal weight. Weighted scores show performance over the full replay distribution, while macro scores check whether improvements are not limited to high-frequency topics.

Overall accuracy and pollution rate are computed over all ordinary replay turns:

$$\text{Accuracy} = \frac{|\{t : \hat{\tau}_t = \tau_t\}|}{N}, \quad \text{Pollution Rate} = 1 - \text{Accuracy}, \quad (4.3)$$

where N is the number of ordinary replay turns. A lower pollution rate means fewer responses were contaminated by another topic.

Topic Recall Matrix. The topic recall matrix evaluates content retention rather than only topic-label selection. After the 309 ordinary replay turns, the dataset adds summarization probes for every topic with at least three user turns. Each probe asks the model to summarize what was discussed about one topic:

$$\text{Probe}(\tau_i) = \text{SUMMARIZE_TOPIC}(\tau_i). \quad (4.4)$$

For each probed topic, the reference trace R_i is built from the user messages and assistant responses that occurred under that topic during the same evaluation run. The model’s generated summary S_i is compared with R_i using ROUGE-1 F1, ROUGE-L F1, and BLEU-2:

$$\text{Recall}^m = \frac{1}{|\mathcal{P}|} \sum_{\tau_i \in \mathcal{P}} m(S_i, R_i), \quad (4.5)$$

where $\mathcal{P}$ is the set of 43 probed topics and m is one of the three overlap metrics. ROUGE-1 measures unigram content overlap, ROUGE-L measures ordered sequence overlap, and BLEU-2 measures short phrase overlap (?, ?, ?). These probes are relevant because a system may output the correct topic prefix while still failing to recover the earlier details of that topic.

In the baseline, all recall probes are asked in the same main node after the full interleaved conversation. The model must therefore recover one topic from a mixed history. In SubchatTrees, probes are routed to the corresponding topic branch when such a branch exists. Since each branch stores a more focused local buffer, summary, and vector memory, the recall matrix directly tests whether topic-specific structure improves long-range retention.

System Performance Matrix. The system performance matrix measures the operational cost of producing correct answers. For each ordinary replay turn, we record input tokens, output tokens, total tokens, and latency. The average total-token cost is:

$$\bar{T}_{\text{total}} = \frac{1}{N} \sum_{t=1}^N T_t. \quad (4.6)$$

Token compression compares the average total tokens used by Subchat Trees against the linear baseline:

$$\text{Token Compression} = \frac{\bar{T}_{\text{baseline}} - \bar{T}_{\text{SubchatTrees}}}{\bar{T}_{\text{baseline}}} \times 100\%. \quad (4.7)$$

Because a cheaper answer is not useful if it is wrong, we also report tokens per

correct answer:

$$\text{Tokens Per Correct Answer} = \frac{\sum_{t=1}^N T_t}{|\{t : \hat{\tau}_t = \tau_t\}|}. \quad (4.8)$$

This combines context accuracy and token use into one efficiency measure. Cost per query is estimated from average input and output tokens:

$$\text{Cost Per Query} = \frac{\overline{T}_{\text{in}} p_{\text{in}} + \overline{T}_{\text{out}} p_{\text{out}}}{10^6}, \quad (4.9)$$

where p_{in} and p_{out} are prices per million input and output tokens.

RAG Decision Matrix. The RAG decision matrix measures how often the system needs to leave the active buffer and retrieve archived context. For every RAG-eligible turn, the retrieval controller returns either *search triggered* or *buffer sufficient*. Let N_{rag} be the number of eligible turns, N_{search} the number of turns that triggered retrieval, and N_{buffer} the number of turns handled by the recent buffer. We report:

$$\text{Retrieval Rate} = \frac{N_{\text{search}}}{N_{\text{rag}}} \times 100\%, \quad \text{Buffer Rate} = \frac{N_{\text{buffer}}}{N_{\text{rag}}} \times 100\%. \quad (4.10)$$

This matrix is relevant because retrieval behavior reflects context locality. In a flat transcript, the active buffer often contains unrelated topics, so the model may need archived retrieval to recover the relevant history. In Subchat Trees, each subchat contains information related mainly to its own topic. A lower retrieval rate together with higher isolation or recall therefore indicates that the system is relying less on archive search because the active branch already contains more relevant context.

4.3 Topic-Scoring Pipeline

All scripted evaluation turns require the model to start its answer with the active topic label followed by a colon. The primary isolation score therefore uses deterministic prefix extraction rather than a subjective semantic judge. This choice makes the metric reproducible and intentionally strict: a response that discusses the correct content but begins with the wrong topic label is counted as polluted because the system failed to preserve conversational state.

For non-introductory turns, the scorer extracts the response prefix using a regular expression of the form `^( [a-zA-Z_][a-zA-Z0-9_]* )\s*:.` . If the extracted prefix equals the normalized ground-truth topic, the turn is counted as correct; otherwise it contributes a false negative for the expected topic and, when applicable, a false positive for the detected topic. Introductory acknowledgement turns are scored only for non-empty completion and are excluded from topic-prefix matching. An LLM-based judge module is retained in the codebase for diagnostic analysis, but the reported isolation tables use the deterministic prefix scorer.

The final evaluation compares a linear-chat baseline against Subchat Trees on the same 309-turn VBRI Dataset. The system evaluate Qwen2.5 7B, 14B, and 32B

Algorithm 2 Deterministic Topic-Prefix Scoring**Require:** Response r , expected topic τ , valid topic set $\mathcal{T}$ **Ensure:** Correctness flag $y \in \{0, 1\}$ and detected topic $\hat{\tau}$

```

1:  $\hat{\tau} \leftarrow \text{REGEXEXTRACTPREFIX}(r)$   $\triangleright$  First token sequence before “:”
2: if  $\hat{\tau} \in \mathcal{T}$  and  $\hat{\tau} = \tau$  then
3:   return  $(1, \hat{\tau})$ 
4: else
5:   return  $(0, \hat{\tau})$ 
6: end if

```

AWQ models at buffer sizes $C \in \{5, 10, 20, 40\}$. The ordinary replay turns measure topic isolation through deterministic prefix scoring. After those turns, 43 recall probes ask the model to summarize prior discussion for each sufficiently frequent topic. This section is organized model by model because the most important result is not a single global score, but how model capacity changes the useful buffer range, retrieval behavior, and failure mode.

4.4 How to Read the Result Tables

The topic-isolation task uses a deliberately strict instruction. The model is told that each new topic starts with a pattern such as `topic_name : user query`, and every response from the LLM active topic name followed by a colon. The key instruction was:

REQUIRED FORMAT for EVERY response:

Start your answer with the active topic or sub-topic name,
followed by a colon, then give your actual answer.

This matters because Table 1 isolation metrics are not semantic relevance scores. A response can discuss the right content and still be counted as polluted if it starts with the wrong topic prefix or no prefix at all. This strict scoring is useful for testing conversational state control. it can also exposes instruction-following limits in smaller models.

On the other hand, The 43 recall probes use a different measurement. For each eligible topic, the probe asks:

Summarize everything we have discussed about {topic}. Include all
key points, details, and questions that were raised.

These probes are not included in topic-prefix F1 score. Instead, they are evaluated using ROUGE-1, ROUGE-L, BLEU-2, probe token cost, and probe RAG decisions. For baseline every probe is asked in a single linear node. In SubchatTrees, probes are routed in their relevant branch. Thus, recall probes measures whether the system can recover topic content after a long interleaving Conversation. On the other hand, isolation F1 tests whether it can correctly classify topic labels based on user query.

4.5 7B Model: Context Structure Helps, but Instruction Following Remains Fragile

The 7B model expose the hardest operating regime. SubchatTrees surplus the baseline at every buffer setting, but the model remains fragile as the context grows. From Table 4.2 it can be observed that system F1 is highest at $C = 5$ (62.9%). As buffer size increases both system and baseliens performance decreases 55.2%, 45.6%, and 39.3%. The baseline even collapses more severely reaching 9.8% F1 at $C = 40$. This is a model specific problem rather than a memory problem. The small model struggle preserving a formatting rule after many turns, as context grows exponentially larger. This is a limitation of the 7B model to follow instruction.

Table 4.2: 7B: merged context-isolation metrics across buffer sizes.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Precision	44.7%	84.0%	71.5%	88.7%	34.9%	80.7%	21.5%	76.2%
Recall	21.1%	58.1%	39.6%	46.1%	15.6%	38.3%	7.8%	33.4%
F1	24.3%	62.9%	44.7%	55.2%	18.7%	45.6%	9.8%	39.3%
Accuracy	21.4%	58.3%	39.8%	46.3%	15.9%	38.5%	8.1%	33.7%
Pollution Rate	78.6%	41.7%	60.2%	53.7%	84.1%	61.5%	91.9%	66.3%
Macro Precision	48.5%	74.4%	70.9%	85.6%	41.7%	76.0%	29.0%	72.8%
Macro Recall	33.7%	59.6%	49.4%	59.7%	24.7%	50.5%	17.8%	41.4%
Macro F1	35.2%	60.8%	53.6%	65.3%	27.2%	55.6%	19.0%	46.5%

The recall table changes the interpretation of the low 7B F1 scores. Table 4.3 shows that the tree system retains more topic content than the baseline. At $C = 20$ ROUGE-1 rises from 0.1811 to 0.4046 double context retention, ROUGE-L rises from 0.1016 to 0.3194 and BLEU-2 rises from 0.0205 to 0.1325. In other words, the subchat system is preserving more useful semantic material than the baseline.

Table 4.3: 7B: aggregate recall-probe scores and probe cost. Topic-specific rows are kept in the appendix; this table reports the major aggregate recall results.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Avg ROUGE-1 (F1)	0.2448	0.4240	0.2369	0.4094	0.1811	0.4056	0.1557	0.3545
Avg ROUGE-L (F1)	0.1270	0.3004	0.1349	0.3097	0.1016	0.3194	0.0861	0.2574
Avg BLEU-2	0.0364	0.1581	0.0324	0.1259	0.0205	0.1325	0.0172	0.1142
Topics Probed	43	43	43	43	43	43	43	43
Total Probe Tokens	158,043	121,521	252,519	188,722	336,942	284,817	472,634	403,852
Avg Tokens per Probe	3,675	2,826	5,873	4,389	7,836	6,624	10,991	9,392
Total Probe Latency	918.00s	710.28s	814.85s	776.91s	767.61s	792.74s	723.47s	703.08s
Avg Probe Latency	21.35s	16.52s	18.95s	18.07s	17.85s	18.44s	16.82s	16.35s

The performance Table 4.3 shows that the system uses more average input token than the baseline. Further, observing the logs reveal that disorganized baseline

context often produces short, generic response. In contrast, Subchat system provides enough context to the model to answer more confidently and fully. This behavior can increase token count. Where suchat provides more detailed and confident response and baseline provide safe and short response. At $C = 10$ to 40 subchat system input becoming slightly larger than the baseline. A better efficiency measurement would be token used per correct answer: SubchatTrees lowers this metrics at every buffer size, including 35,451 to 15,389 at $C = 20$ and at $C = 40$ 102,923 to 25,899

Table 4.4: 7B: merged system-performance metrics, including recall/summarization probe token accounting.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Normal Conversation Turns	309	309	309	309	309	309	309	309
Avg Input Tokens	2,270	2,205	3,384	3,556	5,386	5,685	8,107	8,487
Avg Output Tokens	205	164	214	235	235	241	220	230
Avg Total Tokens	2,476	2,369	3,598	3,791	5,622	5,927	8,327	8,717
Total Tokens	765,017	732,147	1,111,680	1,171,373	1,737,082	1,831,296	2,573,086	2,693,481
Tokens Per Correct Answer	11,591	4,067	9,038	8,191	35,451	15,389	102,923	25,899
Avg Latency	12.19s	10.14s	10.62s	10.55s	11.70s	11.22s	11.75s	11.71s
Cost per Query	\$0.000130	\$0.000123	\$0.000186	\$0.000197	\$0.000288	\$0.000304	\$0.000423	\$0.000443
Summarization Probe Calls	43	43	43	43	43	43	43	43
Summarization Probe Tokens	158,043	121,521	252,519	188,722	336,942	284,817	472,634	403,852
Avg Tokens per Summarization Probe	3,675	2,826	5,873	4,389	7,836	6,624	10,991	9,392
Combined Evaluated Calls	352	352	352	352	352	352	352	352
Combined Total Tokens	923,060	853,668	1,364,199	1,360,095	2,074,024	2,116,113	3,045,720	3,097,333
Combined Avg Tokens per Call	2,622	2,425	3,876	3,864	5,892	6,012	8,653	8,799

The 7B recall-probe RAG table should be read as a retrieval-calibration failure. During recall probes, the system triggers RAG more than the baseline at every buffer size. At $C = 5$, system probe retrieval is 39.5% while baseline probe retrieval is only 2.3%. This is not the expected memory-sufficiency pattern; it suggests the smaller model is unsure whether branch-local context is enough and overuses retrieval despite stronger recall scores. Or toll using capability is limited for this smaller version

Table 4.5: 7B: merged recall/summarization-probe RAG decision breakdown.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Probe Turns	43	43	43	43	43	43	43	43
RAG-Eligible Probe Turns	43	43	43	43	43	43	43	43
Probe RAG Triggered	1	17	2	9	2	4	0	1
Probe Buffer Sufficient	42	26	41	34	41	39	43	42
Probe Retrieval Rate	2.3%	39.5%	4.7%	20.9%	4.7%	9.3%	0.0%	2.3%
Probe Buffer Rate	97.7%	60.5%	95.3%	79.1%	95.3%	90.7%	100.0%	97.7%
Probe Errors	0	0	0	0	0	0	0	0

Overall, the 7B finding is clear: the SubChat architecture improves memory and reduces pollution. Here the main limiting factor was a small model not being able to handle tool calling or follow instruction. The architecture helps, but it cannot fully compensate for weak instruction following.

4.6 14B Model: The Cleanest Practical Tradeoff

The 14B model provides the clearest evidence for the architecture. Table 4.6 shows that system F1 improves from 73.9% at $C = 5$ to 83.4% at $C = 10$ and 83.1% at $C = 20$. The drop at $C = 40$ to 73.1% is also architecturally significant. It shows that increasing the buffer is not automatically beneficial. At $C = 40$ summarization happens later, leaving more raw user and assistant turns in the prompt along with long rolling summaries, system instructions and possible retrieval context. The result is not simply more memory, it is a heavier context that can easily overwhelm the model.

Table 4.6: 14B: merged context-isolation metrics across buffer sizes.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Precision	64.5%	77.8%	81.2%	93.2%	85.0%	89.2%	81.6%	85.3%
Recall	52.6%	74.4%	59.7%	80.2%	60.1%	81.8%	55.8%	69.5%
F1	51.2%	73.9%	64.3%	83.4%	62.5%	83.1%	59.8%	73.1%
Accuracy	52.8%	74.4%	59.9%	80.3%	60.2%	81.9%	56.0%	69.6%
Pollution Rate	47.2%	25.6%	40.1%	19.7%	39.8%	18.1%	44.0%	30.4%
Macro Precision	52.0%	67.5%	75.7%	88.1%	77.6%	85.4%	73.4%	79.5%
Macro Recall	52.7%	68.5%	67.3%	78.7%	65.4%	79.5%	61.8%	65.3%
Macro F1	48.3%	65.8%	67.1%	80.1%	65.1%	79.7%	61.4%	67.9%

The recall-probe table shows the same pattern from the memory side. System ROUGE-1 remains around 0.42 at $C = 10$ and $C = 20$, while baseline ROUGE-1 keeps dropping from 0.3378 at $C = 10$ to 0.2476 at $C = 20$ and 0.1745 at $C = 40$. This in fact indicates that baseline is not only forgetting, it is mixing unrelated context. The tree system keeps the relevant branch recoverable.

Table 4.7: 14B: aggregate recall-probe scores and probe cost.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Avg ROUGE-1 (F1)	0.3428	0.3896	0.3378	0.4201	0.2476	0.4187	0.1745	0.3725
Avg ROUGE-L (F1)	0.2179	0.2634	0.1982	0.3211	0.1320	0.3143	0.0997	0.2648
Avg BLEU-2	0.0967	0.1099	0.0870	0.1338	0.0516	0.1237	0.0182	0.1315
Topics Probed	43	43	43	43	43	43	43	43
Total Probe Tokens	136,045	104,023	274,209	156,136	322,902	269,171	359,355	304,196
Avg Tokens per Probe	3,164	2,419	6,377	3,631	7,509	6,260	8,357	7,074
Total Probe Latency	1155.49s	989.76s	1441.58s	997.15s	1284.58s	1196.07s	943.70s	957.29s
Avg Probe Latency	26.87s	23.02s	33.53s	23.19s	29.87s	27.82s	21.95s	22.26s

The 14B performance table gives the practical tradeoff. Raw input tokens are not always lower for the system, especially at $C = 20$ and $C = 40$, but tokens per correct answer are lower at every buffer size. At $C = 20$, the baseline sends 8,036 tokens per correct answer while the system spends 6,853. At $C = 40$, the baseline sends

10,418 while the system sends 8,826 token. Considering correctness subchat system is more efficient than the baseline linear system.

Table 4.8: 14B: merged system-performance metrics, including recall/summarization probe token accounting.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Normal Conversation Turns	309	309	309	309	309	309	309	309
Avg Input Tokens	1,818	1,919	3,152	3,092	4,659	5,396	5,685	5,981
Avg Output Tokens	133	145	170	182	177	215	148	160
Avg Total Tokens	1,952	2,064	3,321	3,274	4,837	5,611	5,833	6,141
Total Tokens	603,033	637,814	1,026,292	1,011,686	1,494,607	1,733,862	1,802,333	1,897,579
Tokens Per Correct Answer	3,700	2,773	5,548	4,079	8,036	6,853	10,418	8,826
Avg Latency	16.71s	16.16s	15.00s	13.58s	16.06s	17.69s	14.58s	14.64s
Cost per Query	\$0.000102	\$0.000108	\$0.000171	\$0.000169	\$0.000247	\$0.000287	\$0.000296	\$0.000312
Summarization Probe Calls	43	43	43	43	43	43	43	43
Summarization Probe Tokens	136,045	104,023	274,209	156,136	322,902	269,171	359,355	304,196
Avg Tokens per Summarization Probe	3,164	2,419	6,377	3,631	7,509	6,260	8,357	7,074
Combined Evaluated Calls	352	352	352	352	352	352	352	352
Combined Total Tokens	739,078	741,837	1,300,501	1,167,822	1,817,509	2,003,033	2,161,688	2,201,775
Combined Avg Tokens per Call	2,100	2,107	3,695	3,318	5,163	5,690	6,141	6,255

From the RAG table, During recall probes, baseline retrieval is high at small buffers and then decreases as the buffer grows: 58.1%, 44.2%, 37.2%, and 16.3%. The system is much lower: 23.3%, 2.3%, 4.7%, and 0.0%. This behavior is expected. In the baseline, old topic content is either evicted or buried among unrelated turns, so the model asks for archive retrieval. In the tree system, the active branch often already contains enough local evidence, So the LLM decides not to use retrieval. As buffer size grows for both system and baseline the chances of having sufficient context in buffer increase, thus retrieval decreases.

Table 4.9: 14B: merged recall/summarization-probe RAG decision breakdown.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Probe Turns	43	43	43	43	43	43	43	43
RAG-Eligible Probe Turns	43	43	43	43	43	43	43	41
Probe RAG Triggered	25	10	19	1	16	2	7	0
Probe Buffer Sufficient	18	33	24	42	27	41	36	41
Probe Retrieval Rate	58.1%	23.3%	44.2%	2.3%	37.2%	4.7%	16.3%	0.0%
Probe Buffer Rate	41.9%	76.7%	55.8%	97.7%	62.8%	95.3%	83.7%	100.0%
Probe Errors	0	0	0	0	0	0	0	2

The 14B model therefore supports the main architectural claim most cleanly. Moderate buffers provide enough local context while allowing summarization to compact older turns before the prompt becomes too heavy and noisy. In this setting, Subchat Trees further improves isolation, recall, retrieval efficiency and tokens per correct answer.

4.7 32B Model: Strongest Absolute Performance and Better Long-Context Tolerance

The 32B model reaches the strongest absolute performance. Table 4.10 shows that the system is above 83% F1 at only $C = 5$ through $C = 20$. The peak score was 85.2% at $C = 20$. Even at $C = 40$, the subchat system remains at 78.2%, a reasonable drop compare to 7B and 14B models. This is expected as we know stronger models can tolerate longer context better, But the SubChat system still outperform baseline at every buffer settings.

Table 4.10: 32B: merged context-isolation metrics across buffer sizes.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Precision	88.0%	88.5%	87.3%	94.4%	89.9%	93.4%	84.9%	90.0%
Recall	64.6%	83.1%	69.2%	80.2%	67.2%	83.4%	63.6%	74.4%
F1	68.6%	83.2%	71.1%	83.7%	69.6%	85.2%	65.7%	78.2%
Accuracy	64.7%	83.2%	69.3%	80.3%	67.3%	83.5%	63.8%	74.4%
Pollution Rate	35.3%	16.8%	30.7%	19.7%	32.7%	16.5%	36.2%	25.6%
Macro Precision	83.6%	85.1%	81.4%	91.8%	86.7%	93.0%	78.3%	88.2%
Macro Recall	73.9%	81.9%	75.2%	79.8%	75.8%	86.8%	71.6%	74.0%
Macro F1	74.4%	80.8%	73.9%	81.9%	74.5%	87.1%	68.4%	76.7%

The recall metrics 4.11 are also the strongest for the system so far. At $C = 20$ system ROUGE-1 is 0.4283 in compare of 0.3338 for the baseline. As buffer grows, At $C = 40$ the baseline collapses to 0.1503 while the system remains at 0.3918. The reduction in baseline recall happened simply because of too much irrelevant context. This is one of the clearest signs that the linear baseline becomes overwhelmed by topic mixture. The Subchat system keeps the relevant branch recoverable.

Table 4.11: 32B: aggregate recall-probe scores and probe cost.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Avg ROUGE-1 (F1)	0.2687	0.4328	0.3674	0.4206	0.3338	0.4283	0.1503	0.3918
Avg ROUGE-L (F1)	0.1767	0.2830	0.2274	0.2982	0.2009	0.3016	0.0878	0.2624
Avg BLEU-2	0.0502	0.1452	0.1023	0.1253	0.0731	0.1247	0.0234	0.1206
Topics Probed	43	43	43	43	43	43	43	43
Total Probe Tokens	119,832	131,487	249,908	175,875	373,285	264,906	324,171	310,617
Avg Tokens per Probe	2,787	3,058	5,812	4,090	8,681	6,161	7,539	7,224
Total Probe Latency	1959.92s	2495.09s	2965.46s	2722.82s	3002.40s	2796.51s	1888.54s	2288.25s
Avg Probe Latency	45.58s	58.03s	68.96s	63.32s	69.82s	65.04s	43.92s	53.22s

The 32B performance table shows the cost of stronger inference, while confirming the same efficiency pattern observed so far. Though The 32B model is slower than the smaller models but Subchat reduces tokens per correct answer at every buffer

size. At $C = 20$, the baseline sends 7,775 tokens per correct answer but the system sends 6,600. The system also has lower average latency than baseline at $C = 5$, $C = 10$, and $C = 40$. This indicates organized context can offset some of the cost of richer answers.

Table 4.12: 32B: merged system-performance metrics, including recall/summarization probe token accounting.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Normal Conversation Turns	309	309	309	309	309	309	309	309
Avg Input Tokens	2,038	2,119	3,165	3,156	5,043	5,292	6,252	6,405
Avg Output Tokens	139	158	173	176	191	219	162	170
Avg Total Tokens	2,177	2,277	3,338	3,331	5,234	5,511	6,413	6,575
Total Tokens	672,623	703,603	1,031,367	1,029,385	1,617,294	1,702,877	1,981,748	2,031,629
Tokens Per Correct Answer	3,363	2,738	4,819	4,151	7,775	6,600	10,060	8,833
Avg Latency	36.91s	35.86s	37.52s	33.71s	38.99s	39.29s	34.77s	33.56s
Cost per Query	\$0.000113	\$0.000119	\$0.000172	\$0.000172	\$0.000267	\$0.000282	\$0.000326	\$0.000334
Summarization Probe Calls	43	43	43	43	43	43	43	43
Summarization Probe Tokens	119,832	131,487	249,908	175,875	373,285	264,906	324,171	310,617
Avg Tokens per Summarization Probe	2,787	3,058	5,812	4,090	8,681	6,161	7,539	7,224
Combined Evaluated Calls	352	352	352	352	352	352	352	352
Combined Total Tokens	792,455	835,090	1,281,275	1,205,260	1,990,579	1,967,783	2,305,919	2,342,246
Combined Avg Tokens per Call	2,251	2,372	3,640	3,424	5,655	5,590	6,551	6,654

The 32B RAG behavior is more nuanced than the 14B or the 7B model. The system retrieves more during recall probes than the baseline. This suggest a short branch may not contain enough context thus the model decides to use its retrieval tool more often. But as buffer grows 10, 20 and 40, system probe retrieval falls below baseline. At $C = 40$ baseline probe retrieval is 72.1%, meaning it is now more confused with the large mixed context whether the buffer contains relevant information or whether it should search on the vector store with specific keywords that user asked. On the other hand at $C = 40$ Subchat retrieval rate is only 2.4%, indicating the stronger model can use the organized branch context more effectively.

Table 4.13: 32B: merged recall/summarization-probe RAG decision breakdown.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Probe Turns	43	43	43	43	43	43	43	43
RAG-Eligible Probe Turns	43	43	43	43	43	43	43	42
Probe RAG Triggered	23	38	40	23	26	8	31	1
Probe Buffer Sufficient	20	5	3	20	17	35	12	41
Probe Retrieval Rate	53.5%	88.4%	93.0%	53.5%	60.5%	18.6%	72.1%	2.4%
Probe Buffer Rate	46.5%	11.6%	7.0%	46.5%	39.5%	81.4%	27.9%	97.6%
Probe Errors	0	0	0	0	0	0	0	1

The 32B model reveals that The stronger model tolerates larger buffers better, but the tree system still provides a consistent advantage in isolation, recall, pollution reduction and tokens per correct answer.

4.8 Cross-Model Visual Synthesis

The following figures summarize the global pattern across models and buffers. The best observed system performances are reflected in Table 4.14. The best absolute result is 32B at $C = 20$. The 14B at $C = 10/20$ provides a strong practical tradeoff. The 7B model shows the largest rescue effect but instruction-following stays limited.

Table 4.14: Best Subchat Trees configuration for each model.

Model	Best C	Base F1	System F1	F1 Gain	Base Pollution	System Pollution
7B	5	24.3%	62.9%	+38.6 pts	78.6%	41.7%
14B	10	64.3%	83.4%	+19.1 pts	40.1%	19.7%
32B	20	69.6%	85.2%	+15.7 pts	32.7%	16.5%

Figure 4.1 shows context-isolation F1 across all runs. SubchatTrees improves over the baseline in every configuration. The weakest model benefits most in relative terms, while the strongest model reaches the highest absolute performance. SubchatTree shows strong performance, even the 14B Subchat System performs better than the 32B baseline.

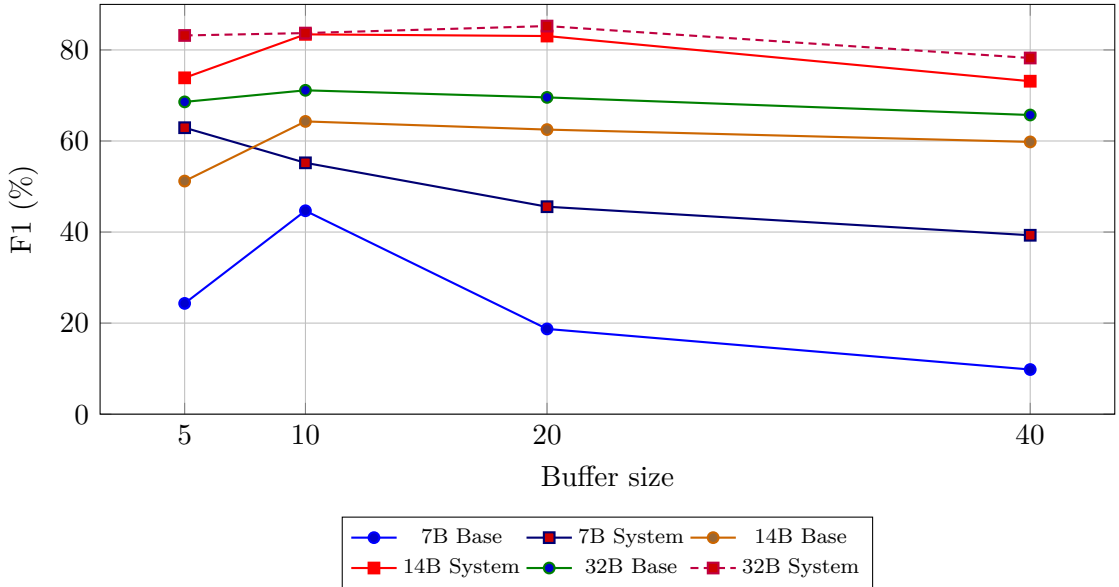


Figure 4.1: Context-isolation F1 across model sizes and buffer sizes. Subchat Trees improves F1 over the linear baseline in every configuration.

Figure 4.2 gives the matching pollution rate. At $C = 20$, pollution falls from 84.1% to 61.5% for 7B, for 14B it is from 39.8% to 18.1%, and 32.7% to 16.5% for 32B. This directly supports that the failure is not only forgetting, but also contamination from unrelated conversational state.

Figure 4.3 summarizes the recall-probe result. The Subchat system improves average ROUGE-1 in every model-buffer setting. This matters because it shows that the tree system is preserving more topic content.

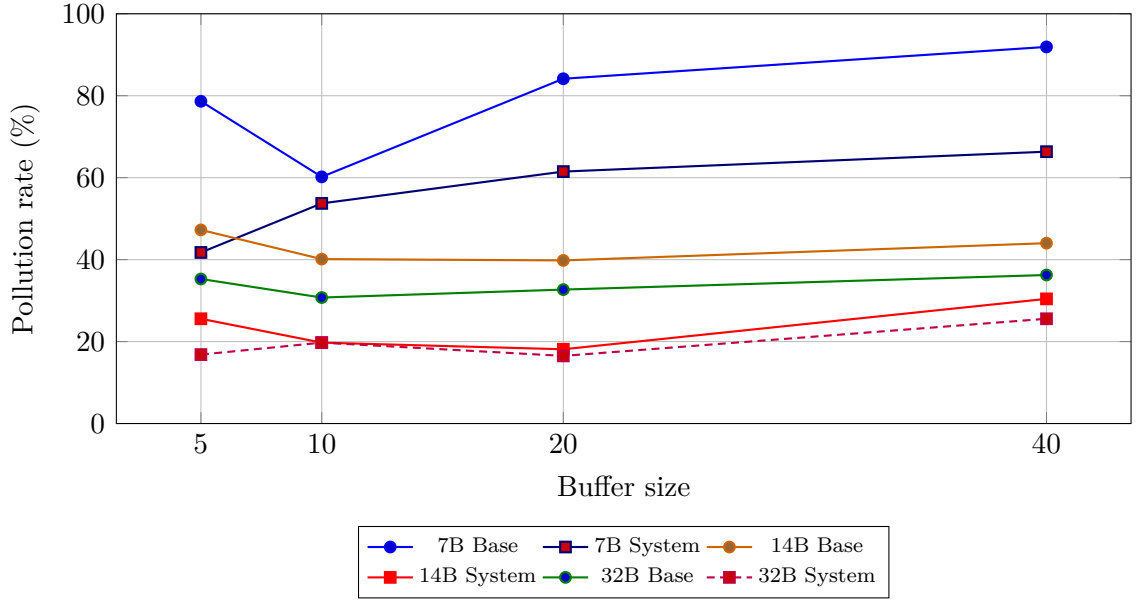


Figure 4.2: Pollution rate across buffer sizes. Lower values are better. Subchat Trees consistently reduces cross-topic contamination by routing turns into branch-local buffers instead of keeping all topics in one linear transcript.

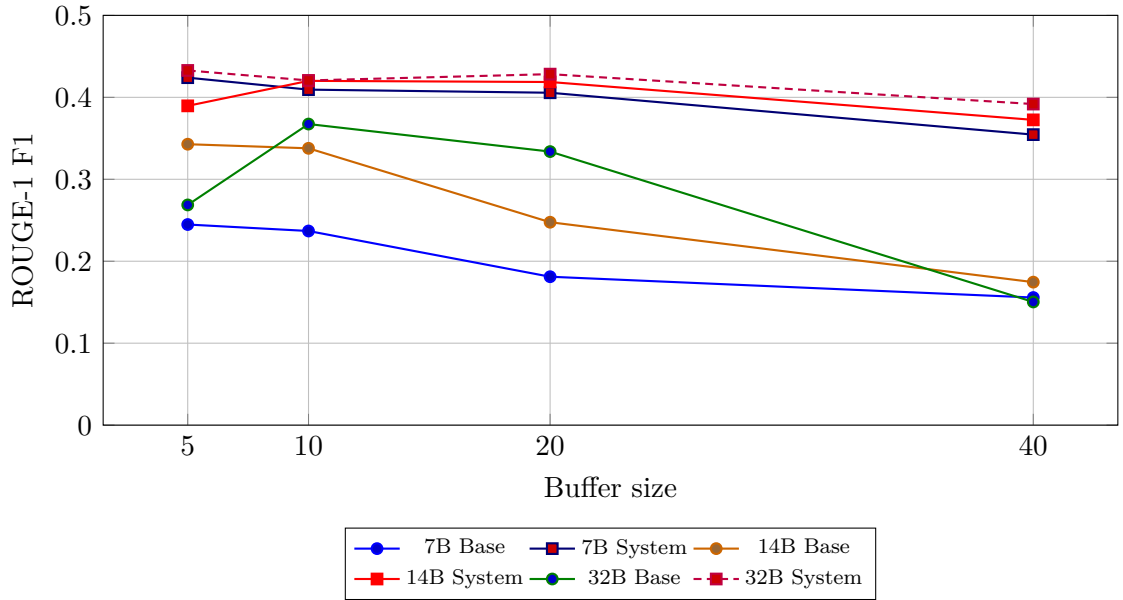


Figure 4.3: Average ROUGE-1 F1 on 43 topic-recall probes. The tree system retains more semantically relevant topic content than the linear baseline across all tested model-buffer settings.

Figure 4.4 focuses on recall-probe RAG decisions. Here each probe explicitly asks the model to recover prior topic content. The 14B model shows the expected pattern most clearly: baseline retrieval is high at small buffers indicating poor context retention. The Subchat System quickly becomes buffer-sufficient. Here, 32B uses more retrieval intentionally indicating small buffer can not store complete context on a topic. As buffer size increases the retrieval rate reduces indicating better context retention in the buffer with less confusion. The model uses more retrieval when the context is unclear or insufficient, But for 7B model because its limited tool calling capability its results are irrelevant.

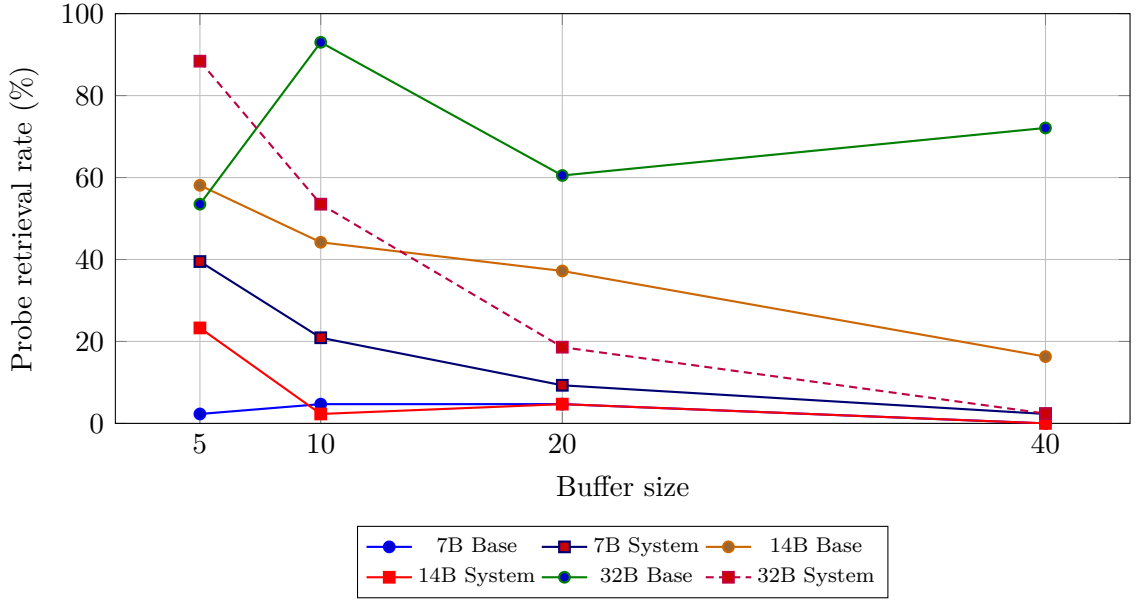


Figure 4.4: RAG retrieval rate during the 43 recall probes. Probe retrieval is used as a memory-sufficiency signal because every probe asks the model to recover earlier topic content.

Finally, Figure 4.5 summarizes effective token efficiency. The system sometimes uses more raw input tokens, but it uses fewer tokens per correct answer in every tested setting. This is the key correction to the initial expectation: organized context is not always shorter, but it is more detailed.

4.9 Result Summary

The model-specific and cross-model results support five conclusions. First, branch-local context improves topic isolation across every tested model and buffer size. Second, the useful buffer range is model-dependent: 7B is safest at short buffers, 14B works best around $C = 10$ or $C = 20$, and 32B peaks at $C = 20$. Third, recall probes show that the tree system preserves semantic topic content even when strict prefix-following fails. Fourth, recall-probe RAG is used as the retrieval diagnostic because each probe explicitly asks for old topic content, the model uses more re-

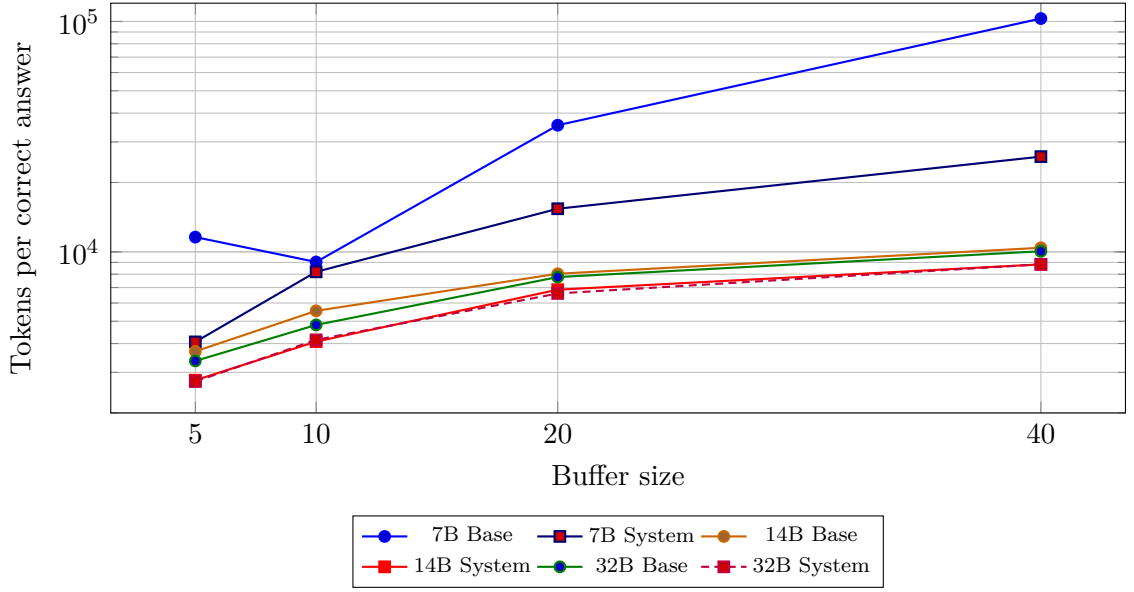


Figure 4.5: Tokens per correct answer across model sizes and buffer sizes. The y-axis uses a log scale because the 7B baseline becomes extremely inefficient at larger buffers.

trieval when context is mixed or insufficient. Fifth, raw input-token count is not the main efficiency result; tokens per correct answer shows that Subchat Trees spends computation more productively as latency for Subchat System was lower.

These results motivate conversation trees as new interface adaption. A parent conversation can hold broad context, child nodes can isolate detailed follow-ups and sibling nodes can explore alternatives without contaminating one another. This matters for ordinary chat, multi-agent workflows and embodied systems where plans, observations, corrections and subgoals naturally form nested conversational state. Long context windows increase capacity, but they do not decide which prior turns belong to the current branch. Subchat Trees makes that structure explicit. It provides more control to the user.

CHAPTER 5

ENGINEERING CONSIDERATIONS, CHALLENGES AND REMEDIES

This chapter summarizes the principal engineering constraints, responsibilities and remedies associated with Subchat Trees.

5.1 Design Constraints

5.1.1 Context and Model Constraints

Fixed context limits require balancing buffer size, summaries and retrieved context. Larger buffers preserve more recent messages but can increase prompt length and pollution; suitable settings therefore depend on model capability.

5.1.2 Memory and Retrieval Constraints

Each node maintains a local buffer and rolling summary, while shared vector memory supports long-term recall. Deduplication, re-ranking and turn-based expansion reduce repeated or fragmented retrieval.

5.1.3 User-Controlled Branching Constraints

Users must create and select appropriate branches. The interface retains conventional chat interaction while exposing optional tree navigation and branch creation.

5.2 Ethical Considerations

The benchmark uses anonymized LMSYS-Chat-1M conversations. Deployments must disclose inherited parent context, support sensitive-data removal and avoid presenting the system as professional advice in high-stakes domains.

5.3 Sustainability in Environmental and Societal Context

Subchat Trees reduces tokens per correct answer, but embeddings, retrieval and summarization add computation. Retrieval should remain conditional, while explicit branches can improve navigation and separation of sensitive or unrelated topics.

5.4 Complex Engineering Problem

The project required the integration of multiple interdependent components and the analysis of model behavior under long, interleaved conversation. Table 5.1 summarizes the complex engineering attributes addressed by this thesis.

Table 5.1: Complex Engineering Problem (CEP)

Name of Complex Engineering Attribute	Justification
P1: Depth of knowledge required	Requires LLM inference, context engineering, hierarchical structures, RAG, vector databases and evaluation.
P3: Depth of analysis required	Compares three model sizes and four buffers across 59 topic labels and 43 recall probes.
P4: Familiarity of issues	Applies known long-context, topic-mixing and retrieval problems to nested user-controlled branches.
P7: Interdependence	Routing, inheritance, buffers, summaries, vector memory, retrieval and inference directly affect one another.

5.5 Engineering Activities

Table 5.2 summarizes the major engineering activities completed during the thesis.

Table 5.2: Complex Engineering Activities (CEA)

Name of Complex Engineering Activity	Justification
A1: Range of Resources	Uses web frameworks, vector memory, inference backends, Kaggle GPUs, GitHub and Qwen2.5 models.
A3: Innovation	Organizes conversation as nested subchats with inherited parent context and isolated child updates.
A4: Consequences for Society and Environment	Improves reliability and privacy while requiring responsible management of token, retrieval and storage costs.

5.6 Development Challenges and Remedies

Long Prompt and vLLM Limits. Multiple buffers were tested and model errors recorded because larger prompts can exceed practical limits.

Context Pollution. Explicit routing, local buffers and summaries separate unrelated topics.

Long-Term Recall. Rolling summaries and vector memory preserve evicted details.

Retrieval Calibration. Probe RAG decisions expose model-dependent retrieval errors.

Reproducibility. Deterministic VBRI replay, fixed settings and recorded configurations support repeatable evaluation.

5.7 Deployment Considerations

Deployment requires vector-index sharding, branch archival, caching and inheritance safeguards. The model-agnostic architecture supports local, cloud and hybrid inference.

CHAPTER 6

DISCUSSION AND CONCLUSION

6.1 Thesis Outcome

This thesis delivers a working hierarchical conversation architecture. Across every tested model and buffer, Subchat Trees improves context-isolation F1, recall retention and tokens per correct answer.

6.2 Thesis Implication and Contribution

The contribution is a user-controlled Subchat Trees architecture with three-tier memory, branch-aware retrieval, the VBRI benchmark and cross-model evaluation. It improves performance without model fine-tuning.

6.3 What the Results Show

Explicit branch routing lowers topic pollution and improves F1 and recall similarity. The 7B model remains limited by instruction following, 14B provides the best practical balance, and 32B achieves the strongest absolute performance at $C = 20$.

6.4 Why Larger Buffers Are Not Always Better

At $C = 40$, delayed summarization and excess raw turns degrade 7B and 14B performance; even 32B performs best at $C = 20$. Longer context increases capacity but does not identify the active conversational branch.

6.5 Token Efficiency Revisited

Subchat Trees can use slightly more raw tokens because relevant context produces fuller answers, while a confused baseline often responds briefly. Tokens per correct answer is therefore the more meaningful measure, and the system is consistently better under it.

6.6 RAG as a Backstop Rather Than a Default

Recall probes show that retrieval works best as a backstop when branch-local context is insufficient. The 7B model is less reliable at judging sufficiency, showing that retrieval control is model-sensitive.

6.7 Limitations

Strict Prefix Scoring. Exact-label scoring can understate semantic quality.

Model Family. Results are limited to Qwen2.5 AWQ models.

Buffer Search. Only $C \in \{5, 10, 20, 40\}$ is evaluated; adaptive sizing may perform better.

RAG Decision Reliability. Weaker models can misjudge context sufficiency.

User-Specified Branching. Real users may organize branches differently from the benchmark.

6.8 Deployment Considerations

Deployment requires scalable vector indexing, archival, caching and inheritance safeguards. The architecture supports local, cloud and hybrid inference.

6.9 Conclusion

Subchat Trees improves context-isolation F1 in every tested configuration. At 32B and $C = 20$, F1 rises from 69.6% to 85.2% and pollution falls from 32.7% to 16.5%; 14B provides a strong practical tradeoff with 83.4% F1 at $C = 10$. Recall and efficiency results further show that long conversations benefit from user-controlled structure, not only longer context windows.

6.10 Future Work

Future work includes adaptive buffers, automatic branch suggestions, controlled cross-node synthesis, multimodal interaction, additional models, human studies and multi-parent graph structures.

6.11 Impact

Subchat Trees addresses critical challenges in conversational AI:

- **Privacy:** Isolated contexts prevent sensitive information leakage across topics.
- **Accuracy:** Reduced pollution improves reliability in high-stakes domains (medicine, law, finance).
- **Cost:** Token efficiency reduces barriers for resource-constrained users and organizations.
- **Accessibility:** Explicit conversation structure aids users in navigating complex, multi-topic discussions.

The implementation, scenarios and evaluation framework support further research in structured conversational AI.

ANNEXURE A

ENGINEERING CONSIDERATIONS, CHALLENGES AND REMEDIES

This annexure summarizes the engineering-compliance evidence required for CSE-400.

A.1 System Requirements and Specifications

The system must inherit parent context, isolate later child messages, preserve evicted information and support different models and buffers.

Table A.1: Functional requirements of Subchat Trees.

ID	Requirement	Implemented Evidence
FR1	Hierarchical branch creation	Create nested subchats from selected parent text.
FR2	Parent-context inheritance	Inherit selected text, recent messages and parent summary.
FR3	Branch isolation	Keep child updates from parent and sibling contexts.
FR4	Bounded short-term memory	Maintain configurable FIFO circular buffers.
FR5	Long-term memory access	Preserve evicted content through summaries and vector memory.
FR6	Conditional retrieval	Retrieve archived blocks only when needed.
FR7	Reproducible evaluation	Replay identical VBRI sequences and record results.

Table A.2: Non-functional requirements and design responses.

ID	Requirement	Design Response
NFR1	Reliability	Local buffers and deterministic replay expose failures.
NFR2	Scalability	Distributed node state and vector memory support longer sessions.
NFR3	Efficiency	Conditional retrieval and tokens-per-correct-answer measure useful work.

ID	Requirement	Design Response
NFR4	Privacy awareness	Isolation reduces leakage; inherited context remains explicit.
NFR5	Usability	Familiar chat interaction with optional tree navigation.
NFR6	Portability	Supports local, cloud and hybrid inference.

A.1.1 Relevant Specifications and Engineering Conventions

The prototype uses UTF-8, structured JSON, version control, FastAPI, Next.js, Server-Sent Events and recorded clean-state experiments. It is not certified for safety-critical deployment.

A.2 Design Constraints

A.2.1 Ethical Considerations

The evaluation uses anonymized public data. Deployments must disclose inherited context, support deletion and avoid replacing professional judgment.

A.2.2 Environmental Considerations

Inference, embeddings and retrieval consume energy. Bounded buffers, reusable embeddings and conditional retrieval reduce avoidable computation.

A.2.3 Social Constraints

Explicit branching can improve navigation but adds cognitive effort. The design therefore preserves familiar chat interaction and treats branching as optional.

A.2.4 Sustainability in Environmental and Societal Context

The project supports SDG 4 through structured learning, SDG 9 through conversational-system innovation and SDG 12 through responsible computational-resource use.

A.3 Project Management

A.3.1 Timeline and Work Breakdown

Table A.3: Project timeline and work breakdown.

Phase	Primary Activity	Deliverable
1	Problem study	Research gap and objectives.
2	Architecture design	Tree, memory and branch workflow.
3	Implementation	Interface, backend and memory pipeline.
4	Evaluation design	VBRI, baseline and metrics.
5	Experiments	Model-buffer runs and logs.
6	Documentation	Analysis, thesis and defense material.

A.3.2 Resource and Budget Management

The prototype used open-source software, existing hardware and available academic or free-tier compute; recorded additional direct expenditure was BDT 0.

Table A.4: Resource and direct-cost budget for the research prototype.

Resource	Direct Cost	Management Decision
Open-source software	BDT 0	Used existing frameworks and libraries.
Experiment compute	BDT 0 additional	Used available Kaggle and API access.
Hardware and storage	BDT 0 additional	Used existing equipment.
Data and version control	BDT 0	Used public data and GitHub.
Recorded additional direct expenditure	BDT 0	Existing resources supported the prototype.

A.3.3 Risk Management

Table A.5: Major project risks and mitigation measures.

Risk	Impact	Mitigation
Model or context-limit failure	High	Test buffers and record errors.
Cross-run state leakage	High	Clear state between runs.
Retrieval miscalibration	Medium	Report probe RAG decisions.
Dataset inconsistency	High	Validate nodes, order and annotations.
Metric misinterpretation	Medium	Separate isolation and semantic recall.

Risk	Impact	Mitigation
Sensitive-context inheritance	in- High	Expose inheritance and support deletion.

A.3.4 Team Roles and Collaboration

Table A.6: Team roles and collaborative responsibilities.

Team Member	Role and Responsibilities
MD Mehedi Hasan	First author; architecture integration, experiments, analysis and thesis preparation.
Kabbo Sarker Surjo	Implementation, testing, validation and review.
Ariful Islam Khan	Interface testing, result checking and review.

Shared source control, logs and review supported collaboration.

A.4 Addressed Knowledge Profile Attributes

Table A.7: Mapping of Knowledge and Attitude Profile attributes.

Attribute	Relevance	Evidence
WK1	Limited	Considers societal implications.
WK2	High	Uses statistics, vectors and quantitative metrics.
WK3	High	Applies data structures and software design.
WK4	High	Uses LLM inference, RAG and dialogue evaluation.
WK5	Medium	Considers token, compute and deployment cost.
WK6	High	Applies web, GPU, database and testing tools.
WK7	Medium	Addresses privacy, reliability and accessibility.
WK8	High	Evaluates a novel conversation architecture.
WK9	Medium	Considers ethics, inclusion and user control.

A.5 Addressed Complex Engineering Problems

The project addresses the following complex engineering characteristics.

Table A.8: Mapping of Complex Engineering Problem attributes.

Attribute	Justification
WP1	Requires advanced LLM, retrieval and evaluation knowledge.
WP2	Balances isolation, recall, usability, cost and privacy.
WP3	Requires an original architecture and benchmark.
WP4	Addresses an uncommon nested-chat problem.
WP5	Operates without a complete established standard.
WP6	Serves users, researchers, developers and operators.
WP7	Integrates interdependent interface, memory, retrieval and inference components.

A.6 Addressed Complex Engineering Activities

Table A.9: Mapping of Complex Engineering Activity attributes.

Attribute	Justification
EA1	Uses data, GPU compute, APIs, databases and version control.
EA2	Resolves interactions among isolation, memory, retrieval, latency and cost.
EA3	Applies trees and memory to user-controlled nested chat.
EA4	Affects privacy, reliability, accessibility and resource use.
EA5	Extends linear chat through a new interaction and memory architecture.

ANNEXURE B

THESIS SELF-ASSESSMENT REPORT

B.1 Research Information

Title of the Research: Subchat Trees: A Scalable Hierarchical Architecture for Multi-Threaded Dialogue and Context Isolation in LLMs

Supervisor: Dr. Md. Aktaruzzaman, Associate Professor and Head, Department of RME, DIU

Co-supervisor: Not applicable

Research Team: MD Mehedi Hasan Moon (first author), Kabbo Sarker Surjo and Ariful Islam Khan

B.2 Synopsis of the Research

Subchat Trees organizes mixed linear chat into user-controlled branches. Each node uses a local buffer, rolling summary and shared vector memory; children inherit parent context without contaminating parent or sibling branches. Across 309 turns, 59 topics, 43 recall probes, three models and four buffers, the architecture improves context-isolation F1 in every tested configuration without fine-tuning.

B.3 Mapping of Attained Course Outcomes

Table B.1: Mapping of attained CSE-400 course outcomes.

Term	CO	Attained Outcome	PO	Level	Relevant Evidence
Spring	CO1	Identified context pollution in linear multi-topic chat as a contemporary computing problem and proposed a research contribution.	PO12	H	Sections 1.2, 2.6, 6.2
Spring	CO2	Identified system outcomes, functional requirements, constraints and specifications for hierarchical conversation management.	PO2	H	Section 1.4, Chapter 3, Annexure A
Spring	CO3	Decomposed the system, managed resources and risks, and documented a timeline and budget.	PO11	H	Sections 3.1–3.5, Annexure A
Spring	CO4	Investigated model behavior through controlled experiments, quantitative metrics and log analysis.	PO4	H	Sections 4.1–4.4
Spring	CO5	Designed, implemented and evaluated the proposed architecture against a linear baseline.	PO3	H	Chapters 3 and 4
Fall	CO1	Applied modern analysis, development and validation tools to the system and subsystems.	PO5	M	Sections 3.5 and 4.1
Fall	CO2	Assessed social impacts and responsibilities related to privacy, accessibility, reliability and resource use.	PO6	M	Section 5.3, Annexure A
Fall	CO3	Evaluated ethical and professional implications of inherited context and high-stakes use.	PO8	H	Section 5.2, Annexure A
Fall	CO4	Considered environmental sustainability through compute use, retrieval decisions and token efficiency.	PO7	L	Section 5.3, Annexure A
Fall	CO5	Worked as a coordinated team using shared implementation, experiments, validation and documentation.	PO9	M	Annexure A, Declaration
Fall	CO6	Communicated the project through the thesis, diagrams, evaluation tables and presentation-oriented findings.	PO10	H	Chapters 3–6, Annexure B

B.4 Assessment Questions and Answers

i. Real-world problem, research gap and future contribution

The project addresses context pollution in linear multi-topic chat through user-controlled branches, branch-aware memory and a structured evaluation benchmark.

ii. Requirements, constraints and functional specifications

Requirements cover inheritance, isolation, bounded buffers, summarization and retrieval under context, cost, usability and privacy constraints.

iii. Planning, scheduling, resource allocation and risk management

The work followed six phases from problem study to documentation, using available resources and controls for model failure, leakage, dataset errors and sensitive inheritance.

iv. Investigation and analysis methods

Deterministic replay, buffer ablation and cross-model experiments compare identical messages using isolation, recall, token, latency, RAG and log evidence.

v. Design, implementation and evaluation

The architecture combines hierarchical nodes, local buffers, rolling summaries, shared vector memory and isolated child updates. Evaluation against a linear baseline shows higher F1 in every configuration.

vi. Modern engineering and computing tools

The project uses FastAPI, Next.js, ChromaDB, SentenceTransformers, vLLM, model APIs, Kaggle GPUs and GitHub for modular implementation and reproducible testing.

vii. Societal impacts and stakeholder responsibilities

Branches can improve accessibility and reduce leakage, while developers remain responsible for transparent inheritance, privacy controls and safeguards.

viii. Ethical and professional issues

Key ethical issues are privacy, inherited sensitive context and high-stakes reliance. Production systems should expose inheritance, support deletion and state their limitations.

ix. Environmental sustainability

Bounded buffers, conditional retrieval, reusable embeddings and suitable models can reduce unnecessary computation; the prototype does not measure full carbon impact.

x. Individual roles and team collaboration

MD Mehedi Hasan Moon coordinated architecture, experiments, analysis and writing. Kabbo Sarker Surjo and Ariful Islam Khan contributed implementation, testing, validation and review.

xi. Communication to technical and non-technical audiences

Technical communication uses the architecture, benchmark, metrics and implementation; non-technical communication explains how branches provide the right context for each task.

B.5 Signature Panel

Dr. Md. Aktaruzzaman
Supervisor
Date:

MD Mehedi Hasan Moon
Student No. 202214048
Date:

Kabbo Sarker Surjo
Student No. 202214030
Date:

Ariful Islam Khan
Student No. 202214039
Date: